



Appunti di Basi di Dati

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

1	Modello Entità-Relazione	10
1.1	Introduzione alla Progettazione Concettuale	10
1.2	I Costrutti Fondamentali nel Dettaglio	10
1.2.1	Le Entità	10
1.2.2	Le Relazioni (o Associazioni)	10
1.2.3	Gli Attributi e Identificatori	10
1.3	Le Cardinalità: Guida alla Lettura	11
1.3.1	Cardinalità Minima	11
1.3.2	Cardinalità Massima	11
1.3.3	Esempio di Analisi Logica	11
1.4	Esempi Pratici e Diagrammi TikZ	11
1.4.1	Esempio 1: Modello Base Studente-Corso (Relazione Moli-a-Molti)	11
1.4.2	Esempio 2: Identificazione Esterna (Entità Debole)	12
1.4.3	Esempio 3: Gerarchia di Generalizzazione (ISA)	12
1.5	Metodologia Pratica di Progettazione	13
2	Il Modello Relazionale in Pratica	14
2.1	Concetti Base: Tabelle, Tuple e Attributi	14
2.2	Schema e Istanza	14
2.3	I Valori NULL	15
2.4	Vincoli di Integrità: Mantenere i dati coerenti	15
2.4.1	Vincoli Intra-relazionali (su singola tabella)	15
2.4.2	Chiavi: Candidata e Primaria	15
2.5	L'anima del Database Relazionale: Le Chiavi Esterne	16
2.6	Dalla Teoria alla Pratica: Esempi di Traduzione ER	16
2.6.1	Esempio 1: Associazione Multi-a-Molti (N:N)	16
2.6.2	Esempio 2: Associazione Uno-a-Molti (1:N)	17
2.6.3	Esempio 3: Lo Storico e le Generalizzazioni	17
3	Algebra Relazionale	18
3.1	Cosa è?	18
3.2	Struttura base di una relazione	18
3.3	Chiavi e Vincoli	18
3.4	Categorie di operatori	19
3.4.1	Precedenza e parentesizzazione	19
3.5	Unione $\cap$	20

3.5.1	Definizione	20
3.5.2	Esempio	20
3.6	Intersezione $\cap$	20
3.6.1	Definizione	20
3.6.2	Esempio	20
3.7	Differenze	20
3.7.1	Definizione	20
3.7.2	Esempio	21
3.8	Ridenominazione ρ	21
3.8.1	Definizione	21
3.8.2	Esempio	21
3.9	Selezione σ	21
3.9.1	Definizione	21
3.9.2	Semantica	22
3.9.3	Esempio	22
3.10	Proiezione π	22
3.10.1	Definizione	22
3.10.2	Esempio	22
3.11	Prodotto cartesiano $\times$	23
3.11.1	Definizione	23
3.11.2	Esempio	23
3.12	Theta-join $\bowtie_{\theta}$	23
3.12.1	Definizione	23
3.12.2	Esempio	23
3.13	Natural-join $\bowtie$	24
3.13.1	Definizione	24
3.13.2	Esempio	24
3.14	Alberi di esecuzione	25
3.14.1	Definizione	25
3.14.2	Esempio	25
3.15	Divisione $\div$	25
3.15.1	Definizione	25
3.15.2	Esempio	25
3.16	Aggregazione (Y)	26
3.16.1	Definizione	26
3.16.2	Esempio	26
4	DDL - Data Definition Language	27
4.1	Notazione	27
4.2	Schemi	27

4.2.1	Cosa sono?	27
4.2.2	Sintassi di creazione	27
4.2.3	Esempio: Creazione di uno schema vuoto	27
4.2.4	Esempio: Creazione di uno schema utilizzando tutte le opzioni disponibili	27
4.3	Tabelle	28
4.3.1	Cosa sono?	28
4.3.2	Sintassi di creazione	28
4.3.3	Esempio: Creazione di una tabella con elementi minimi	28
4.3.4	Esempio: Creazione di una tabella full optional	28
4.4	Definizione dei Dati: I Domini	29
4.4.1	Cosa sono?	29
4.4.2	Domini elementari	29
4.5	Domini definiti dagli utenti	29
4.5.1	Cosa sono?	29
4.5.2	Sintassi di creazione	29
4.5.3	Esempio: Creazione di dominio semplice	29
4.5.4	Esempio: Creazione di un dominio full optional	30
4.6	Vincoli intrarelazionale	30
4.6.1	Tipi di vincoli intrarelazionali	30
4.6.2	Esempio	30
4.7	Vincoli interrelazionali: Integrità referenziale	31
4.7.1	Cosa sono?	31
4.7.2	Sintassi	31
4.7.3	Esempio	31
4.8	Aggiornamento	32
4.8.1	Il problema dell'aggiornamento	32
4.8.2	Il problema delle violazioni	32
4.8.3	Reazioni a violazioni sulla tabella padre	32
4.8.4	Vincoli con nomi	33
4.9	Modifiche degli schemi: ALTER	33
4.9.1	A cosa serve?	33
4.9.2	Tipi di ALTER	33
4.9.3	Sintassi ALTER DOMAIN	33
4.9.4	Esempio di ALTER DOMAIN	33
4.9.5	Sintassi ALTER TABLE	34
4.9.6	Esempio di ALTER TABLE	34
4.10	Vincolo interrelazionale: UPDATE	34
4.10.1	A cosa serve	34
4.10.2	Esempi pratici	35

4.11	Vincolo interrelazionale: DROP	36
4.11.1	Cosa fa?	36
4.11.2	DROP DOMAIN	36
5	Operatore SELECT - Interrogazione di un database	38
5.1	A cosa serve?	38
5.2	Sintassi	38
5.3	Esempio	38
5.4	Istruzione AS	38
5.4.1	A cosa serve?	38
5.4.2	Sintassi	38
5.4.3	Esempio	38
5.5	Clausola FROM	39
5.5.1	Sintassi	39
5.5.2	Notazione punto	39
5.5.3	Esempio	39
5.6	Clausola WHERE	39
5.6.1	A cosa serve?	39
5.6.2	Operatori di confronto	39
5.6.3	Operatore LIKE	39
5.6.4	Operatore BETWEEN	40
5.6.5	Operatore IN	40
5.6.6	Predicati semplici	40
5.6.7	Precedenza di valutazione tra operatori booleani	40
5.6.8	Sintassi	40
5.6.9	Esempio generico	40
5.6.10	Esempio con LIKE	40
5.6.11	Esempio con BETWEEN	41
5.6.12	Esempio con IN	41
5.7	Clausola DISTINCT	41
5.7.1	A cosa serve?	41
5.7.2	Sintassi	41
5.7.3	Esempio	41
5.8	Espressioni	41
5.8.1	A cosa servono?	41
5.8.2	Funzioni predefinite	42
5.8.3	Funzioni per stringhe	42
5.8.4	Esempio di utilizzo di funzioni aritmetiche	42
5.8.5	Esempio di utilizzo di funzioni per stringhe	42

6	Operatore JOIN	43
6.1	A cosa serve?	43
6.1.1	Cosa è un predicato join?	43
6.2	Sintassi JOIN Legacy	43
6.2.1	Esempio	43
6.3	Sintassi JOIN SQL-2	43
6.3.1	Esempio	44
6.4	OUTER JOIN	44
6.4.1	Cosa fa?	44
6.4.2	LEFT OUTER JOIN (o LEFT JOIN)	44
6.4.3	RIGHT JOIN	44
6.5	Self Join	45
6.5.1	Cosa è?	45
6.5.2	A cosa serve?	45
6.5.3	Sintassi	46
6.5.4	Esempio	46
6.6	Join Esplicito	46
6.6.1	Cosa è?	46
6.6.2	A cosa serve?	46
6.6.3	Sintassi	46
6.6.4	Esempio	46
7	Ordinamento dei risultati	47
7.1	ORDER BY	47
7.1.1	Cosa è?	47
7.1.2	Sintassi	47
7.1.3	Esempio	47
7.2	Operatori di aggregazione	47
7.2.1	Quali sono?	47
7.2.2	Come vengono applicati?	48
7.3	COUNT	48
7.3.1	Cosa fa?	48
7.3.2	Sintassi	48
7.3.3	Esempio	48
7.4	SUM	48
7.4.1	Cosa fa?	48
7.4.2	Sintassi	49
7.4.3	Esempio	49
7.5	MIN/MAX	49
7.5.1	Cosa fa?	49

7.5.2	Sintassi	49
7.5.3	Esempio	49
7.6	AVG	49
7.6.1	Cosa fa?	49
7.6.2	Sintassi	49
7.6.3	Esempio	50
7.7	Target List	50
7.8	Operatore di aggregamento GROUP BY	50
7.8.1	Cosa è?	50
7.8.2	Sintassi	50
7.8.3	Esempio	50
7.9	Condizioni WHERE e HAVING	50
7.9.1	Sintassi WHERE	51
7.9.2	Esempio di uso di WHERE	51
7.9.3	Sintassi HAVING	51
7.9.4	Esempio di uso di HAVING	51
8	Interrogazioni di tipi insiemistico	52
8.1	Operatori insiemistici	52
8.1.1	Sintassi	52
8.2	UNION	52
8.2.1	Cosa fa?	52
8.2.2	Sintassi	52
8.2.3	Esempio	52
8.3	INTERSECT	53
8.3.1	Cosa fa?	53
8.3.2	Sintassi	53
8.3.3	Esempio	53
8.4	EXCEPT	53
8.4.1	Cosa fa?	53
8.4.2	Sintassi	54
8.4.3	Esempio	54
9	Interrogazioni Nidificate	55
9.1	Cosa sono?	55
9.2	Come funzionano?	55
9.3	Sintassi Subquery	55
9.4	Predicati ANY e ALL	55
9.4.1	Sintassi	55
9.4.2	Esempio	56

9.5	Quantificatori esistenziali	56
9.5.1	EXISTS	56
9.5.2	NOT EXISTS	56
10	Viste	58
10.1	Cosa sono?	58
10.2	Proprietà fondamentali	58
10.3	A cosa servono?	58
10.4	Sintassi	58
10.5	Esempio creazione vista	59
11	La Progettazione Logica in Pratica	60
11.1	Ristrutturazione dello Schema E-R	60
11.1.1	Analisi delle Ridondanze	60
11.1.2	Eliminazione degli Attributi Multivalore	60
11.1.3	Eliminazione delle Gerarchie (Generalizzazioni)	60
11.2	Traduzione verso il Modello Relazionale	61
11.2.1	Traduzione delle Entità	61
11.2.2	Traduzione delle Relazioni Molti-a-Molti (N:N)	61
11.2.3	Traduzione delle Relazioni Uno-a-Molti (1:N)	62
11.2.4	Traduzione delle Relazioni Uno-a-Uno (1:1)	62
12	Esercizi MR	63
12.1	Totale Giugno 2026	63
12.1.1	Chiavi primarie e vincoli di integrità referenziale	63
12.1.2	Due vincoli intrarelazionali di dominio, uno di tupla.	64
12.1.3	Chiave alternativa per la relazione FILIALE e DIPENDENTE	64
12.1.4	Superchiave per la relazione PACCO	64
13	Esercizi AR	65
13.1	Totale Giugno 26	65
13.1.1	Esercizio 1	65
13.1.2	Esercizio 2	65
13.2	Totale Giugno 2017	66
13.2.1	Es 1	66
13.2.2	Es 2	66
13.2.3	Es 3	66
13.3	Totale Settembre 2015	67
13.3.1	Es 1	67
13.3.2	Es 2	67
13.3.3	Es 3	67

13.4	Slide AR1	68
13.4.1	Es 1	68
13.4.2	Es 2	68
13.4.3	Es 3	68
13.4.4	Es 4	68
13.4.5	Es 5	69
13.5	Slide AR2	70
13.5.1	Es 1	70
13.5.2	Es 2	70
13.5.3	Es 3	70
13.5.4	Es 4	70
13.5.5	Es 5	70
14	Esercizi SQL	71
14.1	Totale Giugno 2026	71
14.1.1	Query 1	71
14.1.2	Query 2	71
14.1.3	Query 3	71
14.1.4	Query 4	72
14.2	Totale Giugno 2015	73
14.2.1	Query 1	73
14.2.2	Query 2	73
14.2.3	Query 3	74
14.3	2° Parziale 2016	75
14.3.1	Query 1	75
14.3.2	Query 2	75
14.3.3	Query 3	75
14.4	Totale Gennaio 2017	76
14.4.1	Query 1	76
14.4.2	Query 2	76
14.4.3	Query 3	76
14.5	Totale Luglio 2017	77
14.5.1	Query 1	77
14.5.2	Query 2	77
14.5.3	Query 3	77
14.6	Esercizio 1	78
14.6.1	Query 1	78
14.6.2	Query 2	78
14.6.3	Query 3	78
14.7	Esercizio 2	79

14.7.1 Query 1	79
14.7.2 Query 2	79
14.7.3 Query 3	79
14.7.4 Query 4	79
14.7.5 Query 5	79
14.8 Esercizio 3	81
14.8.1 Query 1	81
14.8.2 Query 2	81
14.9 Esercizio 4	82
14.9.1 Query 1	82
14.9.2 Query 2	82
14.10 Esercizio 5	83
14.10.1 Query 1	83
14.10.2 Query 2	83
14.10.3 Query 3	83
14.10.4 Query 4	84
14.11 Esercizio 6	85
14.11.1 Query 1	85
14.11.2 Query 2	85
14.11.3 Query 3	85
14.12 Esercizio 7	86
14.12.1 Query 1	86
14.12.2 Query 2	86
14.12.3 Query 3	86
14.13 Esercizio 8	87
14.13.1 Query 1	87
14.13.2 Query 2	87
14.13.3 Query 3	87
14.13.4 Query 4	87
14.13.5 Query 5	87
14.14 Esercizio 9	88
14.14.1 Query 1	88
14.14.2 Query 2	88
14.14.3 Query 3	88

1. Modello Entità-Relazione

1.1 Introduzione alla Progettazione Concettuale

La progettazione di una base di dati si articola in tre fasi fondamentali: la progettazione concettuale, la progettazione logica e la progettazione fisica. Il **Modello Entità-Relazione (E-R)** è lo standard de facto utilizzato nella fase di **progettazione concettuale**.

L'obiettivo principale di questa fase è descrivere la struttura dei dati in modo formale e indipendente dalle scelte tecnologiche (ovvero dal DBMS specifico che verrà adottato). Esso funge da ponte di comunicazione tra i committenti (gli utenti finali) e i progettisti del software.

1.2 I Costrutti Fondamentali nel Dettaglio

1.2.1 Le Entità

Le **entità** rappresentano classi di oggetti del mondo reale (oggetti fisici, persone, concetti astratti o eventi) che possiedono proprietà comuni e un'esistenza autonoma.

- **Regola Pratica:** Nel testo dei requisiti si individuano partendo dai *sostantivi*.
- **Istanze:** Un'entità descrive una classe (es. **Studente**), mentre un'istanza è uno specifico oggetto di quella classe (es. "*Mario Rossi, matricola 123456*").

1.2.2 Le Relazioni (o Associazioni)

Le **relazioni** rappresentano legami logici stabili tra due o più entità.

- **Regola Pratica:** Si individuano partendo dai *verbi* che collegano i sostantivi.
- **Attributi sulle relazioni:** Una relazione può avere essa stessa degli attributi. Questo accade tipicamente nelle relazioni multi-a-molti ($N : M$), dove l'informazione non appartiene a nessuna delle due entità singolarmente, bensì alla loro unione logica.

1.2.3 Gli Attributi e Identificatori

Gli **attributi** descrivono le proprietà elementari delle entità o delle relazioni. Ogni entità deve possedere almeno un **identificatore univoco** (chiamato anche chiave concettuale), composto da uno o più attributi, che permette di distinguere univocamente ogni istanza dell'entità.

- **Attributo Composto:** Un attributo che può essere suddiviso in sotto-componenti (es. Indirizzo composto da *Via*, *Civico*, *CAP*).
- **Identificazione Esterna (Entità Deboli):** Un'entità si dice debole se le sue istanze non possono essere identificate solo tramite i propri attributi interni, ma richiedono il supporto di un'entità forte a cui sono legate (es. un **Modulo d'esame** identificato da un numero progressivo interno e dal **Corso** di afferenza).

1.3 Le Cardinalità: Guida alla Lettura

Le cardinalità indicano quante volte una singola istanza di un'entità può partecipare a una determinata relazione. Viene espressa tramite una coppia di valori (**minimo**, **massimo**) posizionata sul ramo che collega l'entità al rombo della relazione.

1.3.1 Cardinalità Minima

- **0 (Partecipazione Opzionale):** L'istanza può esistere nel database anche senza prendere parte alla relazione.
- **1 (Partecipazione Obbligatoria):** Ogni istanza dell'entità deve necessariamente essere legata ad almeno un'istanza dell'altra entità.

1.3.2 Cardinalità Massima

- **1:** L'istanza può essere legata al massimo a una sola istanza dell'altra entità.
- **N:** L'istanza può essere legata a un numero arbitrario (molteplici) di istanze dell'altra entità.

1.3.3 Esempio di Analisi Logica

Prendiamo la relazione classica tra **Impiegato** e **Dipartimento** attraverso l'associazione **Afferenza**:

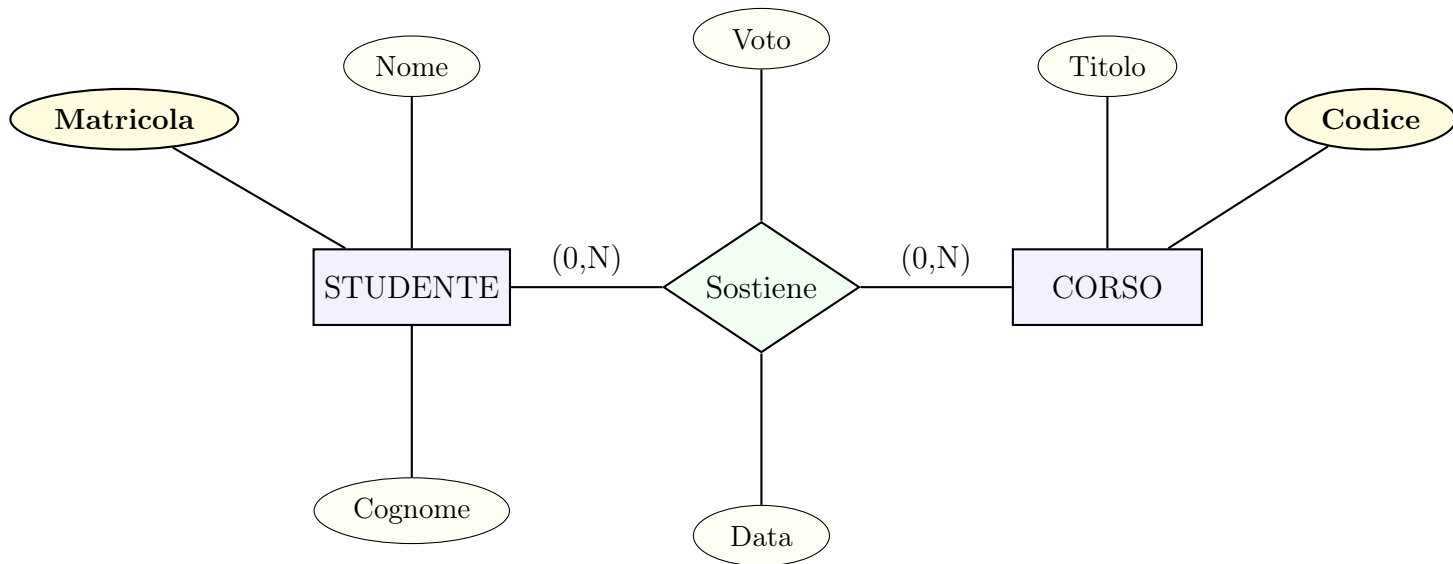
1. Dal punto di vista dell'**Impiegato**: *"Ogni impiegato deve lavorare in uno e un solo dipartimento"*. La cardinalità è **(1,1)**.
2. Dal punto di vista del **Dipartimento**: *"In un dipartimento lavorano da un minimo di 1 impiegato a un massimo di N impiegati"*. La cardinalità è **(1,N)**.

1.4 Esempi Pratici e Diagrammi TikZ

In questa sezione vediamo come implementare i diagrammi graficamente all'interno del documento compilato utilizzando la libreria nativa `tikz`.

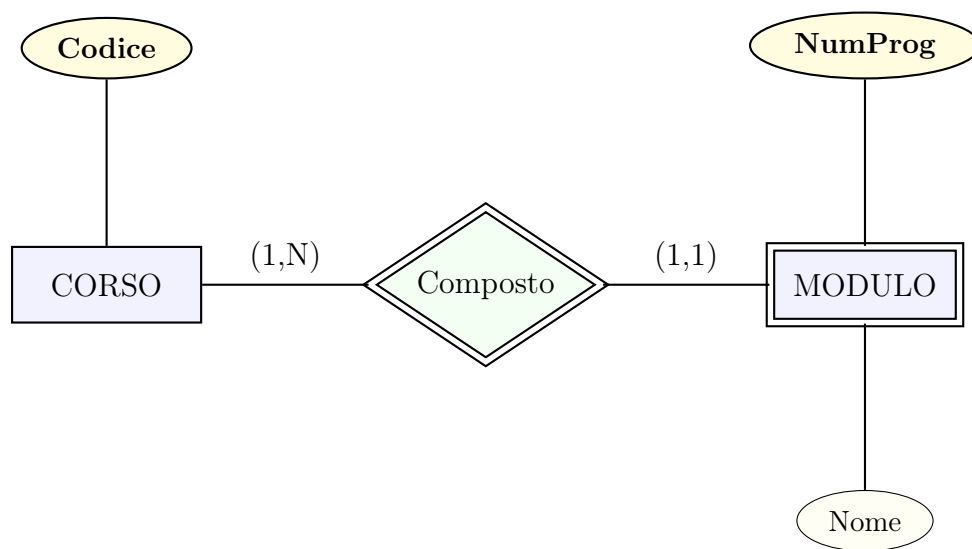
1.4.1 Esempio 1: Modello Base Studente-Corso (Relazione Moli-a-Molti)

Un'applicazione universitaria deve tracciare gli studenti che sostengono gli esami dei rispettivi corsi. Il voto appartiene all'esame sostenuto.



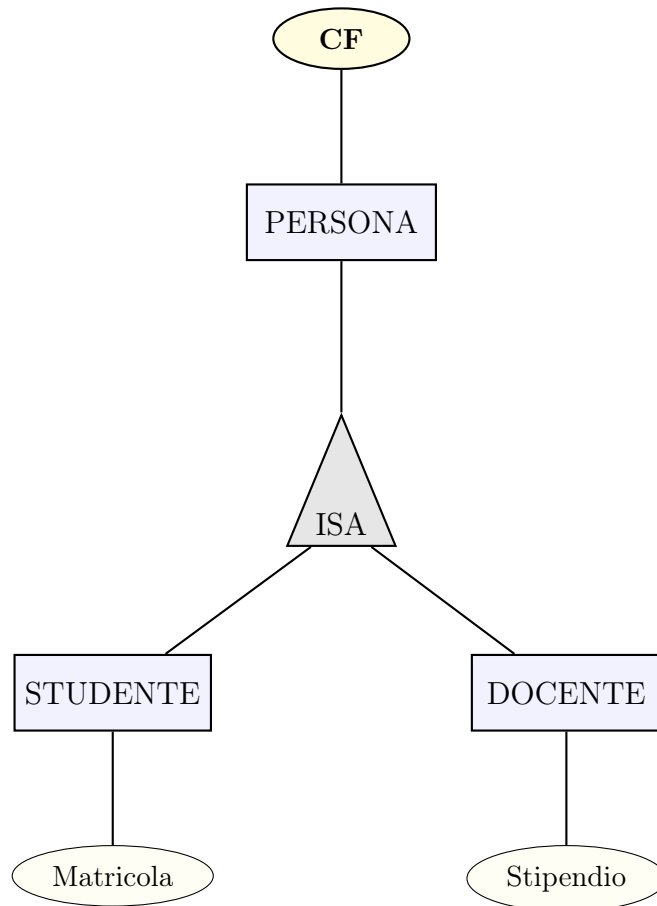
1.4.2 Esempio 2: Identificazione Esterna (Entità Debole)

Un **Corso** universitario è composto da molteplici **Moduli**. Il singolo modulo è identificato da un **NumeroProgressivo** valido solo all'interno di quel corso specifico.



1.4.3 Esempio 3: Gerarchia di Generalizzazione (ISA)

L'entità generica **Persona** viene specializzata nelle sotto-entità **Studente** e **Docente**. Gli attributi comuni rimangono nel padre.



1.5 Metodologia Pratica di Progettazione

Per evitare gli errori più comuni durante gli esami o i progetti aziendali, si consiglia di seguire questi step ordinati:

1. **Costruzione del Glossario dei Termini:** Chiarire i sinonimi e i significati dei vocaboli usati nelle specifiche.
2. **Analisi dei Sostantivi (Entità):** Creare i rettangoli per i concetti dotati di attributi significativi ed esistenza autonoma.
3. **Analisi dei Verbi (Relazioni):** Connettere le entità evitando accuratamente di simulare i collegamenti logici inserendo attributi-puntatore.
4. **Assegnazione delle Cardinalità:** Effettuare un controllo bidirezionale ponendosi sempre le domande al minimo e al massimo per ciascun lato.
5. **Verifica dei Vincoli e delle Ridondanze:** Eliminare attributi derivabili matematicamente o concettualmente tramite cicli nello schema, a meno che non siano esplicitamente giustificati da requisiti di performance.

2. Il Modello Relazionale in Pratica

Il modello relazionale è il pilastro su cui si basano i moderni Data Base Management System (DBMS). A differenza del modello Entità-Relazione (utilizzato per la progettazione concettuale), il modello relazionale ci porta a un livello più vicino alla macchina: la **progettazione logica**. In questo capitolo vedremo come i dati vengano strutturati rigorosamente in tabelle matematiche, dette appunto *relazioni*.

2.1 Concetti Base: Tabelle, Tuple e Attributi

Nel modello relazionale, l'informazione è organizzata in **tabelle** a doppia entrata.

- **Relazione (Tabella)**: Rappresenta un'entità o un'associazione del mondo reale.
- **Attributo (Colonna)**: Rappresenta una proprietà della relazione. Ogni attributo ha un nome e un *dominio* (il tipo di dati ammessi, ad es. interi, stringhe, date).
- **Tupla (Riga)**: Rappresenta un singolo record o istanza della relazione. L'insieme delle tuple forma il contenuto effettivo della tabella in un certo istante.
- **Grado**: Il numero di attributi (colonne) di una relazione.
- **Cardinalità**: Il numero di tuple (righe) presenti in quel momento.

Esempio Pratico: La tabella Studenti

Immaginiamo di voler memorizzare le informazioni degli studenti universitari:

Matricola	Nome	Cognome	DataNascita
1001	Mario	Rossi	15/05/2000
1002	Luca	Bianchi	22/11/1999
1003	Giulia	Verdi	10/01/2001

In questo esempio:

- Il **grado** è 4 (Matricola, Nome, Cognome, DataNascita).
- La **cardinalità** è 3 (ci sono 3 studenti).

2.2 Schema e Istanza

È fondamentale distinguere tra la struttura del database e i dati che esso contiene nel tempo.

- **Schema (Intensione)**: È la struttura logica della tabella, definita dal progettista. Rappresenta la componente *invariante nel tempo*. Si indica generalmente con il nome della tabella seguito dai suoi attributi tra parentesi.

Esempio: STUDENTE(Matricola, Nome, Cognome, DataNascita)

- **Istanza (Estensione):** È l'insieme effettivo delle righe (valori) in un preciso momento. Rappresenta la componente *variabile nel tempo* (aggiornata da inserimenti, modifiche, cancellazioni).

2.3 I Valori NULL

Cosa succede se non conosciamo un'informazione o se questa non si applica al contesto? Nel modello relazionale si utilizza un marcatore speciale chiamato **NULL**. Non rappresenta né il numero zero né una stringa vuota, ma indica esplicitamente "Valore Sconosciuto" oppure "Valore Inapplicabile".

Esempio: Se un lavoratore non ha un numero di telefono aziendale assegnato, il suo attributo **Telefono** assumerà il valore NULL.

Regola d'oro: Nessun attributo che fa parte di una **Chiave Primaria** può assumere valore NULL (questo è noto come *Vincolo di Integrità dell'Entità*).

2.4 Vincoli di Integrità: Mantenere i dati coerenti

Un database non è solo un contenitore, ma un "guardiano" della validità delle informazioni. A questo scopo si usano i **vincoli**.

2.4.1 Vincoli Intra-relazionali (su singola tabella)

1. **Vincoli di Dominio:** Specificano il range o l'insieme di valori permessi per una singola cella.

Esempio: L'attributo **Voto** di un esame deve per forza essere compreso tra 18 e 30 (più l'eventuale lode).

2. **Vincoli di Tupla (o di ennupla):** Esprimono condizioni matematiche o logiche sui valori di una stessa riga, coinvolgendo spesso più colonne.

Esempio: In una tabella **CONTRATTO**(ID, DataInizio, DataFine), deve sempre valere il vincolo $\text{DataFine} \geq \text{DataInizio}$.

2.4.2 Chiavi: Candidata e Primaria

Per garantire che non vi siano righe identiche all'interno di una tabella (poiché, matematicamente, una relazione è un insieme di elementi distinti), si usa il concetto di chiave.

- **Superchiave:** Un insieme di uno o più attributi che identifica in modo univoco una riga.
- **Chiave Candidata:** Una superchiave *minimale*. Non ha attributi in più rispetto allo stretto necessario. Ad esempio, per uno studente, sia la **Matricola** che il **CodiceFiscale** potrebbero fungere da chiavi candidate.
- **Chiave Primaria (Primary Key - PK):** Tra tutte le chiavi candidate, è quella scelta dal progettista come identificatore ufficiale e principale della tabella. Per convenzione, si sottolinea.

Esempio di Schema relazionale con Chiave Primaria:

IMPIEGATO(CodiceFiscale, Nome, Cognome, DataAssunzione)

2.5 L'anima del Database Relazionale: Le Chiavi Esterne

Come colleghiamo due tabelle separate? Attraverso le **Chiavi Esterne** (Foreign Key - FK). Una chiave esterna è un attributo all'interno di una tabella il cui valore deve necessariamente esistere come chiave primaria di un'altra tabella collegata (o, in alternativa, essere NULL).

Esempio Pratico: Impiegati e Dipartimenti

Vogliamo modellare il fatto che gli impiegati lavorano presso specifici dipartimenti.

Schema Logico (l'asterisco indica la chiave esterna):

- DIPARTIMENTO(CodDipartimento, NomeDip, Sede)
- IMPIEGATO(Matricola, Nome, Cognome, CodDip*)

Istanza della tabella DIPARTIMENTO:

CodDipartimento	NomeDip	Sede
D1	Amministrazione	Roma
D2	IT	Milano
D3	Vendite	Napoli

Istanza della tabella IMPIEGATO:

Matricola	Nome	Cognome	CodDip*
100	Alice	Neri	D2
101	Bob	Marroni	D1
102	Carlo	Gialli	D2

Vincolo di Integrità Referenziale: La regola vitale qui è che non posso inserire un nuovo impiegato assegnandogli un dipartimento inesistente (es. `CodDip = 'D9'`). Il DBMS bloccherà l'inserimento generando un errore di integrità referenziale.

2.6 Dalla Teoria alla Pratica: Esempi di Traduzione ER

Vediamo come le cardinalità dei diagrammi Entità-Relazione (ER) influenzano la scrittura del modello relazionale.

2.6.1 Esempio 1: Associazione Multi-a-Molti (N:N)

Scenario: Gli studenti possono seguire molti corsi, e ogni corso è frequentato da molti studenti.

Traduzione Logica: Viene generata una *nuova tabella associativa* (ponte).

- STUDENTE(Matricola, Nome, Cognome)
- CORSO(CodiceCorso, NomeCorso, Crediti)
- ESAME(Matricola*, CodiceCorso*, DataEsame, Voto)

Nota: Nella tabella **ESAME**, la chiave primaria è formata dalla combinazione dei due identificatori (Matricola + CodiceCorso), garantendo che uno studente non possa avere due record per lo stesso esame. Entrambi gli attributi fungono anche da chiavi esterne verso le rispettive tabelle di origine.

2.6.2 Esempio 2: Associazione Uno-a-Molti (1:N)

Scenario: Una libreria possiede molti dipendenti, ma ogni dipendente lavora in una sola libreria (ispirato a un esercizio d'esame tipico).

Traduzione Logica: Non si crea una nuova tabella per l'associazione. La chiave dell'entità lato "1" (Libreria) "migra" come chiave esterna nell'entità lato "N" (Personale).

- `LIBRERIA(idLibreria, indirizzo, citta)`
- `PERSONALE(idPersona, cognome, nome, reparto, idLibreria*)`

2.6.3 Esempio 3: Lo Storico e le Generalizzazioni

Come visto nelle lezioni teoriche, il modello relazionale **non supporta direttamente** l'ereditarietà (il legame padre/figlio). Se hai una generalizzazione nello schema ER (ad esempio, una entità Utente divisa in Docente e Studente), devi risolverla durante la fase di ristrutturazione (prima della traduzione), tipicamente in uno di questi due modi:

- Collassando tutto verso l'alto in un'unica grande tabella **UTENTE** con un attributo **Tipo**.
- Collassando tutto verso il basso generando tabelle distinte **DOCENTE** e **STUDENTE**.

In entrambi i casi, alla fine la struttura deve consistere in semplici relazioni (tabelle) piatte, governate da rigidi legami di chiave primaria ed esterna.

3. Algebra Relazionale

3.1 Cosa è?

L'algebra relazionale è un linguaggio procedurale basato su concetti di tipo algebrico, le operazioni sono definite su relazioni e producono a loro volta altre relazioni.

Si compone di vari operatori fondamentali, che permettono di comporre operazioni in espressioni e alberi di esecuzione per integrazioni anche molto complesse

3.2 Struttura base di una relazione

Una relazione è un'astrazione matematica che definisce un insieme di tuple, ciascuna definita su uno schema comune

- **Schema di relazioni** $R(X)$
 - R = Nome della relazione
 - X = Insiemi di attributi $(A_1, A_2, \dots, A_n)$ ciascuno associato a un dominio di valori
- **Tupla:** Istanza di R , ossia un vettore ordinato di valori $(v_1, v_2, \dots, v_n)$ in $R(X)$
- **Dominio:** Insieme di valori leciti per ciascun attributo
- **Cardinalità** $|R|$: Numero di tuple in R
- **Grado** $|X|$: Numero di attributi di R
- **Tabella:** Rappresentazione "intuitiva" di R , dove le righe sono le tuple e le colonne sono gli attributi

3.3 Chiavi e Vincoli

- **Superchiave:** Insieme di attributi $K \subseteq X$ tale che non esistono 2 tuple distinte t_1, t_2 con $t_1[K] = t_2[K]$
- **Chiave:** Superchiave minimale (nessun sottoinsieme proprio di K è superchiave)
- **Chiavi candidate:** Tutte le chiavi possibili, una di queste diventa primaria
- **Vincoli di integrità referenziale:**
 - Si dichiara `FOREIGN KEY(A1, ..., An) REFERENCES(B1, ..., Bn)`
 - Impone che ogni valore non-NULL di $((A_1, \dots, A_n))$ in R appaia in S su $(B_1, \dots, B_n)$
- **Vincoli di tupla:** Espressioni booleane su attributi
- **Vincoli di dominio:** es $0 \leq \text{Voto} \leq 30$

3.4 Categorie di operatori

- **Operatori Insiemistici**(richiedono schemi identici)
 - $\cap$ Unione
 - $\cup$ Intersezione
 - $-$ Differenza
- **Operatori unari**(agiscono su una sola relazione)
 - σ Selezione: Filtra le tuple che soddisfano una condizione (taglio orizzontale)
 - π Proiezione: Estrae un sottoinsieme di attributi, eliminando duplicati (taglio verticale)
 - ρ Ridenominazione: Rinomina relazione e/o attributi per armonizzare schemi
- **Operatori Binari**(combina due relazioni)
 - $\times$ Prodotto cartesiano: Accoppia ogni tupla di R con ogni tupla di S
 - $\bowtie_{\theta}$ Theta-Join: Join con condizione generica
 - $\bowtie$ Natural-Join: Join automatico su attributo omonimi
- **Operatori Derivati**(costruiti via quelli fondamentali)
 - $\div$ Divisione: Usata per query universali

3.4.1 Precedenza e parentesizzazione

Convenzioni di precedenza (senza parentesi)

1. σ, π, ρ (unari)
2. $\bowtie, \times$ (binari "forti")
3. $\cap, \cup$ (insiemistici)

3.5 Unione $\cup$

3.5.1 Definizione

$$R \cup S = \{t | t \in R \text{ AND } t \in S\}$$

- È un'operazione insiemistica: elimina i duplicati
- Richiede schemi identici (stessi attributi e domini)
- In SQL corrisponde a UNION

3.5.2 Esempio

$$R : (\text{ID}, \text{Nome}) = \{(1, \text{Alice}, (2, \text{Bob}))\} \quad (3.1)$$

$$S : (\text{ID}, \text{Nome}) = \{(2, \text{Bob}, (3, \text{Claudia}))\} \quad (3.2)$$

$$R \cup S = \{(1, \text{Alice}, (2, \text{Bob})), (3, \text{Claudia})\} \quad (3.3)$$

3.6 Intersezione $\cap$

3.6.1 Definizione

$$R \cap S = \{t | t \in R \text{ OR } t \in S\}$$

- Trova tutte le tuple comuni
- Richiede ancora schemi identici
- In SQL corrisponde a INTERSECT

3.6.2 Esempio

$$R : (\text{ID}, \text{Nome}) = \{(1, \text{Alice}, (2, \text{Bob}))\} \quad (3.4)$$

$$S : (\text{ID}, \text{Nome}) = \{(2, \text{Bob}), (3, \text{Claudia})\} \quad (3.5)$$

$$R \cap S = \{(2, \text{Bob})\} \quad (3.6)$$

3.7 Differenze

3.7.1 Definizione

$$R - S = \{t | t \in R \text{ AND } t \notin S\}$$

- Non commutativa: $R - S \neq S - R$
- Consente di rimuovere tuple indesiderate
- In SQL corrisponde a EXCEPT (o MINUS)

3.7.2 Esempio

$$R : (\text{ID}, \text{Nome}) = \{(1, \text{Alice}), (2, \text{Bob})\} \quad (3.7)$$

$$S : (\text{ID}, \text{Nome}) = \{(2, \text{Bob}), (3, \text{Claudia})\} \quad (3.8)$$

$$R - S = \{(2, \text{Bob})\} \quad (3.9)$$

3.8 Ridenominazione ρ

3.8.1 Definizione

- $\rho_{B_1, \dots, B_n \leftarrow A_1, \dots, A_n}(R)$ cambia solo i nomi degli attributi $A_i \rightarrow B_i$, lasciando inalterati i valori
- Utili per:
 - Preparare operazioni insiemistiche (schemi compatibili)
 - Disambiguare auto-join
- Se volessimo cambiare anche il nome della relazioni in R' , allora $R' \leftarrow \rho(R)$ oppure $\rho(R) \rightarrow R'$

3.8.2 Esempio

Padre	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele

PATERNITÀ

Madre	Figlio
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

MATERNITÀ

Genitore	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

$$\rho_{\text{Genitore} \leftarrow \text{Padre}}(\text{PATERNITÀ}) \cup \rho_{\text{Genitore} \leftarrow \text{Madre}}(\text{MATERNITÀ})$$

3.9 Selezione σ

3.9.1 Definizione

$$\sigma_F(R)$$

- Filtra tuple in base ad una condizione F
- Predicati:
 - $A \theta B$ oppure $A \theta c$, con $\theta \in \{=, \neq, >, <, \geq, \leq\}$ è un operatore di confronto, A e B sono attributi di su cui il confronto θ abbia un senso, e c'è una costante compatibile con il dominio di A
 - combinati $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT)

3.9.2 Semantica

- Mantiene stesso schema di R
- Risultato: sottoinsieme delle tuple(o righe) di R
- Riduce la cardinalità, dunque abbiamo un'ottimizzazione se la spingiamo in anticipo (cioè, abbiamo meno tuple su cui applicare la sotto-operazione)

3.9.3 Esempio

ID	Età	Reparto
1	28	IT
2	35	HR
3	42	IT

R

ID	Età	Reparto
2	35	HR
3	42	IT

$\sigma_{\text{Età} > 30}(\text{R})$

ID	Età	Reparto
3	42	IT

$\sigma_{\text{Età} > 30 \wedge \text{Reparto} = \text{'IT'}}(\text{R})$

3.10 Proiezione π

3.10.1 Definizione

$$\pi_{A_1, \dots, A_n}(R)$$

- Seleziona un sottoinsieme di attributi (colonne)
- Elimina i duplicati (semantica insiemistica)

3.10.2 Esempio

ID	Età	Reparto
1	28	IT
2	35	HR
3	42	IT

R

Reparto
IT
HR

$\pi_{\text{Reparto}}(\text{R})$

3.11 Prodotto cartesiano $\times$

3.11.1 Definizione

$$R \times S = \{(t, u) | t \in R \wedge u \in S\} \quad (\text{concatena tuple})$$

- Schema: attributi di R unito agli attributi di S
- Cardinalità: $|R| \cdot |S|$

3.11.2 Esempio

ID	Cliente
1	Andrea
2	Barbara

R

Codice	Prodotto	Quantità
A	Scarpe	100
B	Magliette	200
C	Occhiali	300

S

ID	Cliente	Codice	Prodotto	Quantità
1	Andrea	A	Scarpe	100
1	Andrea	B	Magliette	200
1	Andrea	C	Occhiali	300
2	Barbara	A	Scarpe	100
2	Barbara	B	Magliette	200
2	Barbara	C	Occhiali	300

$R \times S$

3.12 Theta-join $\bowtie_{\theta}$

3.12.1 Definizione

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

- θ è una qualsiasi condizione su attributi di $R \cup S$
- Quando la condizione θ è una congiunzione di atomi di uguaglianza, il theta-join viene chiamato equi-join
- Il self-join si ha quando facciamo la join tra una relazione con se stessa

3.12.2 Esempio

Impiegato	Progetto
Rossi	A
Neri	A
Neri	B

IMPIEGATI

Codice	Nome
A	Venere
B	Marte

PROGETTI

Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	B	Marte

$$\text{IMPIEGATI} \bowtie_{\text{Progetto=Codice}} \text{PROGETTI} = \sigma_{\text{Progetto=Codice}}(\text{IMPIEGATI} \times \text{PROGETTI})$$

3.13 Natural-join $\bowtie$

3.13.1 Definizione

$$R(A, B, C) \text{ e } S(B, C, D) \Rightarrow R \bowtie S = \pi_{A,B,C,D} (R \bowtie_{B=B' \wedge C=C'} (\rho_{B',C' \leftarrow B,C}(S)))$$

- Join su tutti gli attributi con lo stesso nome
- Elimina automaticamente le colonne duplicate

3.13.2 Esempio

Impiegato	Reparto
Rossi	Vendite
Neri	Produzione
Verdi	Produzione

R

Reparto	Capo
Produzione	Mori
Vendite	Bruni

S

Impiegato	Reparto	Capo
Rossi	Vendite	Bruni
Neri	Produzione	Mori
Verdi	Produzione	Mori

$R \bowtie S$

3.14 Alberi di esecuzione

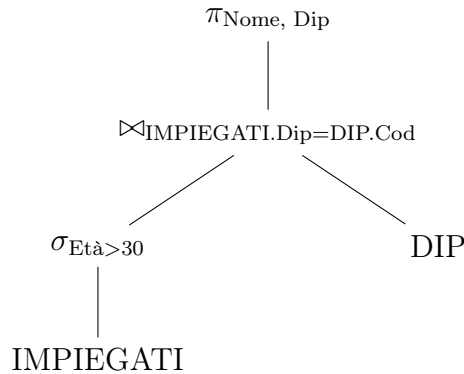
3.14.1 Definizione

Un'espressione si rappresenta come un albero

- Le **foglie** sono relazioni di base
- I **nodi interni** sono operatori di base

3.14.2 Esempio

L'espressione $\pi_{\text{Nome}, \text{Dip}}(\sigma_{\text{Età} > 30}(\text{IMPIEGATI}) \bowtie_{\text{IMPIEGATI.Dip} = \text{DIP.Cod}} \text{DIP})$ ha il seguente albero di esecuzione:



3.15 Divisione $\div$

3.15.1 Definizione

$$R \div S = \{t[X] \mid t \in R \text{ e } \forall s \in S (t[X], s[Y]) \in R\}$$

- **Schema ricorrente:** $R(X, Y)$ e $S(Y) \implies$ risultato sui soli attributi di X
- La divisione si può derivare da altri operatori.
In effetti, siano $R(X, Y)$ e $S(Y)$ due relazioni allora:

$$R \div S = \pi_X((\pi_X(R) \times S) - R)$$

È un operatore non primitivo, ma utile in scrittura

3.15.2 Esempio

$$\pi_{\text{CodImp}}(\text{IMPIEGATO} \div \text{PROG_OBBLIGATORIO})$$

Risultato: Gli impiegati che compaiono con ogni CodProg in PROG_OBBLIGATORIO

3.16 Aggregazione (Y)

3.16.1 Definizione

$$Y_{G;f1(A1) \rightarrow F1, \dots}(R)$$

- G sono gli attributi di raggruppamento
- Le funzioni f sono funzioni di aggregazione (COUNT,SUM,MAX,MIN,AVG)
- $f1(A1) \rightarrow F1$ vuol dire la funzione f1 applica all'attributo A1 e rinomina il risultato come F1

3.16.2 Esempio

- **Schema:**

IMPIEGATO(CodImp, Dip, Stip)

- **Richiesta:** trovare gli stipendi medi per ogni dipartimento
- **Espressione:** $\gamma_{Dip;AVG(Stip) \rightarrow Stip_medio}(IMPIEGATO)$
- **Risultato:** una tupla per Dip con colonna Stip_medio

CodImp	Dip	Stip
1	1	10000
2	1	20000
3	2	5000
4	2	3000
5	2	7000

IMPIEGATO

Dip	Stip_medio
1	15000
2	5000

$\gamma_{Dip;AVG(Stip) \rightarrow Stip_medio}(IMPIEGATO)$

4. DDL - Data Definition Language

4.1 Notazione

- **TERMINI DEL LINGUAGGIO** in maiuscolo
- `<x>` usato per isolare un termine x
- `[x]` indicano che il termine x è opzionale
- `{x}` indicano che il termine x può essere ripetuto da 0 a un numero arbitrario di volte
- `|` separa opzioni alternative

4.2 Schemi

4.2.1 Cosa sono?

Uno schema di base di dati è una collezione di oggetti: domini, tabelle, asserzioni, viste, privilegi, ...

4.2.2 Sintassi di creazione

```
CREATE SCHEMA
  [NomeSchema] -- Se omissso e' il nome dell'utente che ha lanciato il
               comando
  [[authorization] Autorizzazione] -- Termine del linguaggio
  {DefinizioneElementoSchema} -- Termine variabile
```

Dopo il comando **CREATE SCHEMA** compaiono le definizioni dei vari elementi.

Non è necessario che la definizione di tutti gli elementi avvenga contemporaneamente alla creazione dello schema, può avvenire in più fasi successive

4.2.3 Esempio: Creazione di uno schema vuoto

```
CREATE SCHEMA GestioneAzienda;
```

4.2.4 Esempio: Creazione di uno schema utilizzando tutte le opzioni disponibili

```
CREATE SCHEMA GestioneAzienda -- Nome dello schema
AUTHORIZATION utente_admin -- Autorizzazione
CREATE TABLE Impiegato ( -- Termine variabile
  id_impiegato INT PRIMARY KEY,
  nome VARCHAR(50),
```

```

    cognome VARCHAR(50)
)
CREATE VIEW VistaImpiegati AS -- Altro termine variabile
SELECT nome, cognome FROM Impiegato;

```

4.3 Tabelle

4.3.1 Cosa sono?

Una tabella è costituita da una collezione ordinata di attributi e da un insieme (eventualmente vuoto) di vincoli

4.3.2 Sintassi di creazione

```

CREATE TABLE NomeTabella(
    NomeAttributo DOMINIO [ValoreDiDefault][Vincoli]
    {NomeAttributo DOMINIO [ValoreDiDefault][Vincoli]}
    [AltriVincoli]
)

```

4.3.3 Esempio: Creazione di una tabella con elementi minimi

```

CREATE TABLE Studente (
    matricola INT PRIMARY KEY,
    nome VARCHAR(30)
);

```

4.3.4 Esempio: Creazione di una tabella full optional

```

CREATE TABLE Esame (
    codice_esame INT,
    nome_corso VARCHAR(50) NOT NULL,
    data_esame DATE DEFAULT CURRENT_DATE,
    voto INT CHECK (voto BETWEEN 18 AND 30),
    matricola_studente INT,

    -- Definizione della chiave primaria composta
    CONSTRAINT PK_Esime PRIMARY KEY (codice_esame, matricola_studente),

    -- Definizione della chiave esterna con vincolo di integrita
    referenziale
    CONSTRAINT FK_Studente FOREIGN KEY (matricola_studente)
        REFERENCES Studente(matricola)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

```

4.4 Definizione dei Dati: I Domini

4.4.1 Cosa sono?

I domini specificano i valori ammessi di ciascun attributo

4.4.2 Domini elementari

- **Testuali**

- **varchar(n)**: Stringa di lunghezza variabile tra 0 e n caratteri
- **char(n)**: Stringa di lunghezza pari ad n

- **Numerici**

- **Numerici Esatti**

- * **integer** / **smallint** : rappresentano valori interi
- * **numeric** / **decimal** : rappresentano i valori decimali

- **Numerici approssimati**

- * **real** / **double precision** : rappresentano valori a singola / doppia precisione in virgola mobile
- * **float** permette di richiedere la precisione che si desidera

- **boolean**: un booleano

- **blob, clob**: oggetti di grandi dimensioni

- costruiti da binari (blob)
- costruiti da caratteri (clob)

vengono trattati e memorizzati diversamente dagli altri, vengono usati per informazioni non strutturate o multimediali

4.5 Domini definiti dagli utenti

4.5.1 Cosa sono?

Sono dei domini definiti dall'utente partendo dai domini elementari

4.5.2 Sintassi di creazione

```
CREATE DOMAIN NomeDominio AS DominioElementare  
    [ValoreDefault]  
    [Constraints]
```

4.5.3 Esempio: Creazione di dominio semplice

```
CREATE DOMAIN Voto AS INT;
```

4.5.4 Esempio: Creazione di un dominio full optional

```
CREATE DOMAIN Voto AS SMALLINT
DEFAULT '0'
CHECK (value >= 18 AND value <= 30);
```

4.6 Vincoli intrarelazionale

Un vincolo intrarelazione è una regola che specifica delle condizioni all'interno di una relazione

4.6.1 Tipidi di vincoli intrarelazionali

Proprietà sempre valida all'interno di una relazione

- **NOT NULL:** Il valore deve essere non nullo
- **UNIQUE:** I valori devono essere non ripetuti
- **PRIMARY KEY:** Chiave primaria (una sola per tabella)
- **CHECK:** Definisce condizioni complesse (sia intra che inter relazionali)

4.6.2 Esempio

```
CREATE TABLE Dipendenti (
  -- 1. VINCOLO DI CHIAVE PRIMARIA (e di dominio NOT NULL implicito)
  ID_Dipendente INT PRIMARY KEY,

  -- 2. VINCOLI DI DOMINIO (Tipo dato, NOT NULL e lunghezza)
  Nome VARCHAR(50) NOT NULL,
  Cognome VARCHAR(50) NOT NULL,

  -- 3. VINCOLO DI CHIAVE UNICA (Unique)
  -- L'email deve essere unica per ogni dipendente, ma potrebbe essere
  -- temporaneamente assente (NULL)
  Email VARCHAR(100) UNIQUE,

  -- 4. VINCOLO DI DOMINIO con controllo sul range (CHECK)
  -- Lo stipendio deve essere un numero positivo
  Stipendio_Mensile DECIMAL(10, 2) CHECK (Stipendio_Mensile > 0),

  Data_Assunzione DATE NOT NULL,
  Data_Fine_Contratto DATE,

  -- 5. VINCOLO DI TUPLA (CHECK che coinvolge piu colonne)
  -- La data di fine contratto deve essere successiva alla data di
  -- assunzione
  CONSTRAINT CHK_Date_Contratto CHECK (Data_Fine_Contratto >
    Data_Assunzione)
);
```

4.7 Vincoli interrelazionali: Integrità referenziale

4.7.1 Cosa sono?

Esprimono un legame gerarchico (padre / figlio) fra tabelle.

Alcuni attributi della tabella figlio sono definiti **FOREIGN KEY** si devono riferire (**REFERENCES**) ad alcuni attributi della tabella padre che costituiscono una chiave.

I valori contenuti nella **FOREIGN KEY** devono essere sempre presenti nella tabella padre.

4.7.2 Sintassi

REFERENCES

Nella parte di definizione degli attributi con il costrutto sintattico **REFERENCES**

```
AttrFiglio CHAR(3) REFERENCES TabellaPadre(AttrPadre)
```

FOREIGN KEY

Oppure dopo la definizione degli attributi con i costruttori **FOREGIN KEY** e **REFERENCES**

```
FOREIGN KEY(AttrFiglio)
    REFERENCES TabellaPadre(AttrPadre)
```

Quando si hanno più attributi da riferire, si utilizza sempre **FOREIGN KEY**

```
FOREIGN KEY(AttrFiglio1[, AttrFiglio2])
    REFERENCES TabellaPadre(AttrPadre1[, AttrPadre2])
```

Se si omettono gli attributi destinazione, vengono assunti quelli della chiave primaria

```
AttrFiglio CHAR(3) REFERENCES TabellaPadre
```

4.7.3 Esempio

```
CREATE TABLE Infrazioni (
    Codice INTEGER PRIMARY KEY, -- Identificativo dell'infrazione (es.
    34321)
    Data DATE NOT NULL,
    Vigile INTEGER NOT NULL,
    Prov CHAR(2),
    Numero CHAR(6),

    -- Vincolo interrelazionale singolo (in linea o a livello tabella)
    FOREIGN KEY (Vigile) REFERENCES Vigili(Matricola),

    -- Vincolo interrelazionale composto (obbligatoriamente a fine
    tabella)
    FOREIGN KEY (Prov, Numero) REFERENCES Auto(Prov, Numero)
);
```

4.8 Aggiornamento

4.8.1 Il problema dell'aggiornamento

Studente				Esame			
Matr	Nome	Città	CDip	Matr	CodCorso	Data	Voto
34321	Luca	Mi	Inf	34321	1	25/02/01	28
53524	Giovanni	To	Mat	34321	2	14/12/01	30
64521	Emilio	Ge	Ing	64521	1	12/03/02	24
73321	Francesca	Vr	Mat	73321	4	18/01/02	18

Se elimino da Studente la tupla con matricola 34321, cosa succede alla tabella esame?

4.8.2 Il problema delle violazioni

Se viene eseguita una operazione di aggiornamento che porta alla violazione di un vincolo di integrità referenziale la base di dati diventa non valida.

Per evitare ciò, vengono definite un insieme di politiche per evitare che questo accada, permettendo di scegliere delle reazioni da adottare in caso di violazione.

Queste politiche vanno dichiarate nella dichiarazione DDL dello schema.

Per tutti gli altri tipi di vincoli invece a seguito di una violazione il comando di aggiornamento viene rifiutato segnalando l'errore.

Tipo di violazione

Inserimento o modifica di tupla che viola il vincolo sulla **tabella figlio** (interna)



Politica di reazione

In entrambi i casi il comando non viene eseguito

Inserimento o modifica di tupla che viola il vincolo sulla tabella padre (esterna)



Quattro politiche possibili

4.8.3 Reazioni a violazioni sulla tabella padre

- **CASCADE:** L'operazione di modifica / cancellazione viene propagata alla tabella intera
- **SET NULL:** NULL nella tabella figlio in entrambi i casi
- **SET DEFAULT:** Default nella tabella figlio in entrambi i casi
- **NO ACTION:** Rifiutata in entrambi i casi

4.8.4 Vincoli con nomi

A fini diagnostici è spesso utile sapere quale vincolo è stato violato a seguito di un'azione sul DB. A tale scopo è possibile associare dei nomi ai vincoli, ad esempio:

```
Stipendio INTEGER CONSTRAINT StipendioPositivo  
CHECK (Stipendio > 0)
```

```
CONSTRAINT ForeignKeySedi  
FOREIGN KEY (Sede) REFERENCES Sedi
```

4.9 Modifiche degli schemi: ALTER

4.9.1 A cosa serve?

Modifica oggetti

4.9.2 Tipi di ALTER

- **ALTER DOMAIN:** Modifica domini
- **ALTER TABLE:** Modifica tabelle

Quando si definisce un nuovo vincolo deve essere soddisfatto dalla istanza presente, altrimenti viene rifiutato

4.9.3 Sintassi ALTER DOMAIN

```
ALTER DOMAIN NomeDominio <  
    SET          DEFAULT ValoreDefault |  
    DROP        DEFAULT |  
    ADD         CONSTRAINT DefVincolo |  
    DROP        CONSTRAINT NomeVincolo >
```

4.9.4 Esempio di ALTER DOMAIN

```
-- 1. Definiamo un valore di default standard per tutti i nuovi  
    inserimenti  
ALTER DOMAIN VotoEsame  
    SET DEFAULT 18;  
-- 2. Aggiungiamo un vincolo per gestire anche l'estensione del voto con  
    la "Lode" (es. inserendo 31)  
ALTER DOMAIN VotoEsame  
    ADD CONSTRAINT ControlloloLode CHECK (VALUE >= 18 AND VALUE <= 31);  
-- 3. Rimuoviamo il valore di default inserito in precedenza se non è più  
    necessario  
ALTER DOMAIN VotoEsame  
    DROP DEFAULT;
```

```
-- 4. Rimuoviamo il vincolo sulla lode per tornare alle impostazioni base
ALTER DOMAIN VotoEsame
    DROP CONSTRAINT ControlloLode;
```

4.9.5 Sintassi ALTER TABLE

```
ALTER TABLE NomeTabella <
    ALTER COLUMN NomeAttributo <
        SET DEFAULT NuovoDefault |
        DROP DEFAULT > |
    DROP COLUMN NomeAttributo |
    ADD COLUMN DefAttributo |
    DROP CONSTRAINT NomeVincolo |
    ADD CONSTRAINT DefVincolo >
```

4.9.6 Esempio di ALTER TABLE

```
-- 1. Aggiungiamo una nuova colonna per memorizzare l'Email dello
    studente
ALTER TABLE Studente
    ADD COLUMN Email VARCHAR(100);
-- 2. Modifichiamo la colonna "Citta" esistente per impostare un valore
    di default predefinito
ALTER TABLE Studente
    ALTER COLUMN Citta SET DEFAULT 'Mi';
-- 3. Rimuoviamo la colonna "CDip" (Codice Dipartimento) se la tabella
    viene ristrutturata
ALTER TABLE Studente
    DROP COLUMN CDip;
-- 4. Aggiungiamo un vincolo di unicita (Unique) sulla colonna Email
    appena creata
ALTER TABLE Studente
    ADD CONSTRAINT EmailUnica UNIQUE (Email);
-- 5. Rimuoviamo un vecchio vincolo di controllo (se esistente)
    identificato dal suo nome
ALTER TABLE Studente
    DROP CONSTRAINT VerificaMatricola;
```

4.10 Vincolo interrelazionale: UPDATE

Quando si effettua una modifica (UPDATE) sui dati all'interno di una base di dati relazionale, è fondamentale garantire che i legami tra le tabelle rimangano consistenti. Il vincolo di integrità referenziale (Foreign Key) interviene in modo specifico per regolamentare cosa succede quando viene modificata una chiave.

4.10.1 A cosa serve

Il vincolo interrelazionale durante un'operazione di modifica ha lo scopo di impedire la creazione di "tuple orfane". Si considerano due scenari distinti:

- **Modifica sulla tabella figlio (interna):** Se si modifica il valore della chiave esterna nella tabella figlio, il database controlla semplicemente che il nuovo valore esista nella chiave primaria della tabella padre. Se non esiste, l'operazione viene bloccata.
- **Modifica sulla tabella padre (esterna):** Se si modifica il valore della chiave primaria nella tabella padre, tutti i record della tabella figlio che puntavano a quel vecchio valore rimarrebbero associati a una chiave non più esistente. Per evitare questa violazione, il DBMS applica delle specifiche *politiche di reazione*.

Sintassi dell'operazione

La definizione della politica di reazione per la modifica viene dichiarata in fase di creazione o alterazione della tabella figlio, subito dopo la specifica della chiave esterna (**FOREIGN KEY**).

La sintassi generica all'interno del linguaggio SQL DDL è la seguente:

```
FOREIGN KEY (AttributoTabellaFiglio) REFERENCES TabellaPadre(
    AttributoTabellaPadre) ON UPDATE [ RESTRICT | CASCADE | SET NULL | SET
    DEFAULT ]
```

Il significato delle quattro politiche possibili è il seguente:

1. **RESTRICT (o NO ACTION):** È il comportamento di default. La modifica sulla tabella padre viene rifiutata e solleva un errore se esistono tuple collegate nella tabella figlio.
2. **CASCADE:** La modifica viene propagata a cascata. Modificando il valore nella tabella padre, tutti i record corrispondenti nella tabella figlio verranno aggiornati automaticamente con il nuovo valore.
3. **SET NULL:** Il valore della chiave esterna nei record correlati della tabella figlio viene impostato a NULL (richiede che la colonna non abbia il vincolo NOT NULL).
4. **SET DEFAULT:** Il valore della chiave esterna nella tabella figlio viene reimpostato al valore di default precedentemente configurato per quella colonna.

4.10.2 Esempi pratici

Riprendiamo come riferimento le tabelle **Studente** (tabella padre, con chiave primaria **Matr**) ed **Esame** (tabella figlio, con chiave esterna **Matr** che fa riferimento a **Studente**).

Esempio 1: Politica CASCADE (Propagazione automatica)

Si supponga di voler modificare la matricola dello studente 'Luca' da 34321 a 99999. Configurando la tabella con la politica **CASCADE**:

```
ALTER TABLE Esame
    ADD CONSTRAINT fk_studente
    FOREIGN KEY (Matr) REFERENCES Studente(Mat)
    ON UPDATE CASCADE;
```

Eseguendo l'istruzione `UPDATE Studente SET Matr = 99999 WHERE Matr = 34321;`, il database aggiornerà automaticamente anche tutte le righe all'interno della tabella **Esame** in cui la matricola era 34321, sostituendola con 99999.

Esempio 2: Politica RESTRICT (Blocco cautelativo)

Nel caso in cui le matricole debbano rimanere tassativamente immutabili per ragioni storiche e di sicurezza, si opta per RESTRICT:

```
ALTER TABLE Esame
  ADD CONSTRAINT fk_studente
  FOREIGN KEY (Matr) REFERENCES Studente (Matr)
  ON UPDATE RESTRICT;
```

Se si provasse ad eseguire la modifica della matricola 34321 nella tabella **Studente**, il DBMS bloccherebbe l'operazione restituendo un errore del tipo: *“Violazione del vincolo di chiave esterna: record correlati presenti nella tabella Esame”*.

Esempio 3: Politica SET NULL (Annullamento del legame)

Se l'esame sostenuto deve rimanere registrato nel sistema ma, in seguito alla modifica o riorganizzazione di una targa/codice identificativo padre, si decidesse temporaneamente di scollegarlo dallo studente:

```
ALTER TABLE Esame
  ADD CONSTRAINT fk_studente
  FOREIGN KEY (Matr) REFERENCES Studente (Matr)
  ON UPDATE SET NULL;
```

Cambiando la matricola nella tabella padre, le righe corrispondenti nella tabella **Esame** vedranno il proprio campo **Matr** trasformarsi in NULL, indicando che l'esame è al momento “orfano” di uno studente identificato.

4.11 Vincolo interrelazionale: DROP

4.11.1 Cosa fa?

Cancella oggetti dallo schema

Ha 2 opzioni:

- **RESTRICT:** Un oggetto non è rimosso se non è vuoto
- **CASCADE:** Viene rimosso l'oggetto e tutto ciò che è coinvolto nella definizione dell'oggetto

4.11.2 DROP DOMAIN

Cancellazione del dominio

Sintassi

```
DROP DOMAIN NomeDominio [ RESTRICT | CASCADE ];
```

Esempio di utilizzo

```
-- 1. Creazione del dominio
CREATE DOMAIN CodiceDipartimento AS CHAR(3)
    CHECK (VALUE IS NOT NULL);

-- 2. Utilizzo del dominio nella tabella "Studente"
CREATE TABLE Studente (
    Matr INT PRIMARY KEY,
    Nome VARCHAR(50),
    CDip CodiceDipartimento -- La colonna CDip usa il dominio appena
        creato
);

DROP DOMAIN CodiceDipartimento RESTRICT;
```

DROP TABLE

Cancellazione della tabella

Sintassi

```
DROP TABLE NomeTabella [ RESTRICT | CASCADE ];
```

Esempi

```
-- Creazione della tabella Padre
CREATE TABLE Studente (
    Matr INT PRIMARY KEY,
    Nome VARCHAR(50)
);

-- Creazione della tabella Figlio con il vincolo di Foreign Key
CREATE TABLE Esame (
    CodCorso INT,
    Matr INT,
    Voto INT,
    PRIMARY KEY (CodCorso, Matr),
    CONSTRAINT fk_studente FOREIGN KEY (Matr) REFERENCES Studente(Matrx)
);

DROP TABLE Studente RESTRICT;
-- Nota: Scrivere "DROP TABLE Studente;" produce lo stesso effetto
```

5. Operatore SELECT - Interrogazione di un database

5.1 A cosa serve?

È la istruzione utilizzata per effettuare interrogazioni al database

5.2 Sintassi

```
SELECT ListaAttributi  
FROM ListaTabelle  
[WHERE Condizione]
```

5.3 Esempio

```
SELECT Nome, Cognome, Stipendio  
FROM Impiegato  
WHERE Mansione = 'Ingegnere'
```

5.4 Istruzione AS

5.4.1 A cosa serve?

Viene utilizzato per fare ridenominazioni su attributi e anche su relazioni

5.4.2 Sintassi

```
SELECT AttrEspr [ [ AS ] Alias ] {, AttrEspr [ [ AS ] Alias ] }  
FROM Tabella [ [ AS ] Alias ] {, Tabella [ [ AS ] Alias ] }  
[ WHERE Condizione ]
```

5.4.3 Esempio

```
SELECT I.Nome, D.Citta AS citta_lavoro  
FROM Impiegato AS I, Dip AS D  
WHERE I.Dipart = D.Nome;
```

5.5 Clausola FROM

Specifica la/le tabelle/e su cui operare per determinare il risultato

5.5.1 Sintassi

```
SELECT ListaAttributi  
FROM Tabella1 [ [ AS ] Alias1 ] {, TabellaN [ [ AS ] AliasN ] }
```

5.5.2 Notazione punto

La clausola . permette di specificare a che relazione appartiene un attributo (o anche una tabella a un database)

5.5.3 Esempio

```
SELECT I.Nome , D.NomeDip  
FROM Impiegato AS I, Dip AS D  
WHERE I.Dipart = D.CodDip;
```

5.6 Clausola WHERE

5.6.1 A cosa serve?

È una espressione booleana che specifica una o più condizioni di selezione

5.6.2 Operatori di confronto

- =
- <>
- <= =>
- LIKE
- BETWEEN
- IS NULL
- IS NOT NULL

5.6.3 Operatore LIKE

L'operatore LIKE permette di esprimere dei "pattern" su stringhe di caratteri (regex) mediante la seguente notazione:

- _ indica un singolo carattere arbitrario
- % indica una stringa di caratteri arbitraria (anche lunga 0)

5.6.4 Operatore BETWEEN

L'operatore BETWEEN permette di esprimere condizioni di appartenenza a un intervallo

5.6.5 Operatore IN

L'operatore IN permette di esprimere condizioni di appartenenza a un insieme

5.6.6 Predicati semplici

Sono espressioni di valutazione su attributi mediante operatori di confronto

5.6.7 Precedenza di valutazione tra operatori booleani

1. NOT
2. AND
3. OR

5.6.8 Sintassi

```
WHERE [NOT] PredicatoSemplice {< AND | OR > [NOT] PredicatoSemplice}
```

5.6.9 Esempio generico

```
SELECT Nome, Cognome, Mansione
FROM Impiegato
WHERE ufficio = '10';
```

5.6.10 Esempio con LIKE

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Cognome LIKE '%i';
```

Figura 5.1: Restituire tutti i cognomi che finiscono con la lettera i

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Cognome LIKE 'R_ssi';
```

Figura 5.2: Restituirà "Rossi" (dove _ sostituisce la 'o'), ma non "Rossini" (perché _ impone un solo carattere)

5.6.11 Esempio con BETWEEN

```
SELECT Nome, Stipendio
FROM Impiegato
WHERE Stipendio BETWEEN 38 AND 60;
```

5.6.12 Esempio con IN

```
SELECT Cognome, ufficio
FROM Impiegato
WHERE ufficio IN ('10', '20');
```

5.7 Clausola DISTINCT

5.7.1 A cosa serve?

In SQL, le tabelle prodotte dalle interrogazioni possono contenere più righe identiche tra loro. I duplicati possono essere rimossi usando la parola chiave DISTINCT

5.7.2 Sintassi

```
SELECT DISTINCT ListaAttributi
FROM ListaTabelle
[ WHERE Condizione ]
```

5.7.3 Esempio

```
SELECT DISTINCT Cognome, Citta
FROM Impiegato
WHERE Stipendio > 40;
```

5.8 Espressioni

5.8.1 A cosa servono?

Una espressione usata nella clausola SELECT dà luogo ad una nuova "colonna" che non corrisponde a nessuna colonna della tabella su cui si effettua la selezione.

Questa nuova colonna è detta colonna virtuale.

Le colonne virtuali non sono fisicamente memorizzate, ma esistono solo come risultato delle interrogazioni.

Anche alle colonne virtuali è possibile assegnare un alias (con il costrutto AS)

5.8.2 Funzioni predefinite

SQL mette a disposizione alcune funzioni predefinite:

- **Stringhe:** UPPER, UCASE, LENGTH, ...
- **Date / Intervalli:** +, -, DATE, DAYOFWEEK, ...
- **Matematiche:** *, +, -, /, TAN, SQRT, SIN, ...
- **Informazioni di sistema:** USER, CURRENT_DATE, ...

5.8.3 Funzioni per stringhe

Gli operatori più comuni per le stringhe sono:

- **||:** Operatore di concatenazione
- **UPPER, UCASE:** Trasforma la stringa in caratteri maiuscoli
- **LOWER, LCASE:** Trasforma la stringa in caratteri minuscoli

5.8.4 Esempio di utilizzo di funzioni aritmetiche

```
SELECT Nome, Stipendio, Premioprod, (Stipendio + Premioprod) AS  
       stipendio_totale  
FROM Impiegato  
WHERE Mansione = 'Ingegnere';
```

5.8.5 Esempio di utilizzo di funzioni per stringhe

```
SELECT Nome, Cognome, citta, mansione  
FROM Impiegato  
WHERE UCASE(mansione) = 'INGEGNERE';
```

6. Operatore JOIN

6.1 A cosa serve?

L'operatore JOIN rappresenta un'importante funzionalità in quanto permette di correlare dati in tabelle diverse.

Matematicamente si tratta del prodotto cartesiano tra le 2 tabelle, che avrà un numero di righe pari al prodotto del numero di righe delle due tabelle.

Ad un operatore JOIN sono applicati uno o più predicati join.

6.1.1 Cosa è un predicato join?

Un predicato di join esprime una condizione che deve essere verificata dalle tuple del risultato dell'interrogazione.

Il predicato di join permette di estrarre dal DB un sottoinsieme opportuno del prodotto cartesiano.

6.2 Sintassi JOIN Legacy

Nella sintassi legacy, le tabelle da unire vengono elencate nella clausola FROM separate da una virgola, e la condizione di join viene specificata nella clausola WHERE

```
SELECT ListaAttributi
FROM Tabella1, Tabella2
WHERE Tabella1.Attributo = Tabella2.Attributo;
```

6.2.1 Esempio

```
SELECT Impiegato.Nome, Dip.Citta AS citta_lavoro
FROM Impiegato, Dip
WHERE Impiegato.Dipart = Dip.Nome;
```

6.3 Sintassi JOIN SQL-2

Lo standard SQL-2 ha introdotto una sintassi dedicata che separa chiaramente la logica di unione (la JOIN) dalle condizioni di filtro (la WHERE), rendendo le interrogazioni più leggibili e riducendo il rischio di errori nel dimenticare le clausole di collegamento

```
SELECT ListaAttributi
FROM Tabella1 [AS Alias1]
JOIN Tabella2 [AS Alias2] ON Tabella1.Attributo = Tabella2.Attributo;
```

6.3.1 Esempio

```
SELECT I.Nome, D.Citta AS citta_lavoro
FROM Impiegato AS I
JOIN Dip AS D ON I.Dipart = D.Nome;
```

6.4 OUTER JOIN

6.4.1 Cosa fa?

L'operatore OUTER JOIN è un'estensione della JOIN standard (Inner Join) utilizzata quando si desidera conservare nel risultato anche le tuple che non hanno corrispondenze nell'altra tabella.

Mentre l'Inner Join restituisce solo le righe che soddisfano la condizione di unione (ovvero quelle con corrispondenza in entrambe le tabelle), l'Outer Join garantisce che almeno una delle due tabelle (o entrambe) sia rappresentata completamente.

Esistono 3 tipologie di OUTER JOIN

6.4.2 LEFT OUTER JOIN (o LEFT JOIN)

Definizione

Restituisce tutte le righe della tabella indicata a sinistra, per le righe della tabella che non hanno corrispondenza nella tabella di destra, vengono visualizzati valori NULL per tutti gli attributi provenienti dalla tabella di destra

Sintassi

```
SELECT colonne
FROM TabellaA A
LEFT JOIN TabellaB B ON A.chiave = B.chiave;
```

Esempio

```
SELECT I.Nome, D.Citta
FROM Impiegato I
LEFT JOIN Dip D ON I.Dipart = D.Nome;
```

6.4.3 RIGHT JOIN

Definizione

Restituisce tutte le righe della tabella indicata a destra, per le righe della tabella che non hanno corrispondenza nella tabella di sinistra, vengono visualizzati valori NULL per tutti gli attributi provenienti dalla tabella di sinistra

Sintassi

```
SELECT I.Nome, D.NomeDipartimento
FROM Impiegato I
RIGHT JOIN Dip D ON I.Dipart = D.ID_Dipartimento;
```

Esempio

```
SELECT I.Nome, D.NomeDipartimento
FROM Impiegato I
RIGHT JOIN Dip D ON I.Dipart = D.NomeDipartimento;
```

FULL JOIN

Definizione

La FULL JOIN restituisce tutte le righe provenienti da entrambe le tabelle coinvolte nell'operazione. Quando non esiste una corrispondenza tra una riga di una tabella e l'altra, il risultato includerà comunque la riga, popolando con valori NULL tutti gli attributi provenienti dalla tabella in cui manca il riscontro.

Sintassi

```
SELECT colonne
FROM TabellaA A
FULL JOIN TabellaB B ON A.chiave = B.chiave;
```

Esempio

```
SELECT I.Impiegato, D.Dip
FROM Impiegato I
FULL JOIN Dip D ON I.Dipart = D.Dip;
```

6.5 Self Join

6.5.1 Cosa è?

La Self Join (o auto-congiunzione) è un caso speciale di JOIN in cui una tabella viene congiunta con se stessa.

Non esiste un comando SQL chiamato SELF JOIN: si utilizza la sintassi classica della INNER JOIN o della LEFT JOIN, ma riferendosi alla stessa tabella sia a destra che a sinistra dell'operatore.

6.5.2 A cosa serve?

Serve quando i dati memorizzati in una riga hanno una relazione con i dati memorizzati in un'altra riga della stessa tabella.

6.5.3 Sintassi

```
SELECT <colonne>
FROM <nome_tabella> AS <alias1>
[INNER | LEFT] JOIN <nome_tabella> AS <alias2>
    ON <alias1>.<colonna_connessione> = <alias2>.<colonna_chiave>;
```

6.5.4 Esempio

```
SELECT
    E.Nome AS Impiegato,
    M.Nome AS Manager
FROM Impiegato AS E
LEFT JOIN Impiegato AS M
    ON E.ID_Manager = M.ID;
```

6.6 Join Esplicito

6.6.1 Cosa è?

Il join esplicito è un approccio che definisce l'unione tra tabelle usando un'apposita sintassi dedicata.

6.6.2 A cosa serve?

A livello logico, serve a separare nettamente la struttura di collegamento tra le tabelle dai filtri di selezione sui dati.

La clausola specifica direttamente quali tabelle unire e isola la condizione di join. Questo riduce drasticamente il rischio di errori distruttivi e aumenta la leggibilità del codice. Inoltre, garantisce la flessibilità necessaria a implementare facilmente vari tipi di unione esterna (come LEFT, RIGHT o FULL JOIN).

6.6.3 Sintassi

```
SELECT colonna1, colonna2, ...
FROM TabellaA [AliasA]
JOIN TabellaB [AliasB]
    ON AliasA.chiave_esterna = AliasB.chiave_primaria
[WHERE altre_condizioni_di_filtro];
```

6.6.4 Esempio

```
SELECT I.Nome, D.Citta
FROM Impiegato AS I
JOIN Dip AS D
    ON I.Dipart = D.Nome;
```

7. Ordinamento dei risultati

7.1 ORDER BY

7.1.1 Cosa è?

L'ORDER BY è una clausola opzionale che si inserisce al termina di una query SELECT per ordinare le righe del risultato in base ai valori di una o più colonne

7.1.2 Sintassi

```
SELECT colonna1, colonna2, ...  
FROM NomeTabella  
[WHERE condizioni_di_filtro]  
ORDER BY colonnaA [ASC | DESC], colonnaB [ASC | DESC];
```

Ordinamento

- **Ascendente (ASC)** è default e può essere omesso
- **Discendente (DESC)**

7.1.3 Esempio

```
SELECT  
    UCASE(I.Nome) AS Nome_Impiegato,  
    ((I.Stipendio + I.Premio) * 13) AS Guadagno_Annuo_Stimato,  
    D.Citta AS Sede_Dipartimento  
FROM Impiegato AS I  
JOIN Dip AS D  
    ON I.Dipart = D.Nome  
WHERE UCASE(I.Mansione) = 'INGEGNERE'  
ORDER BY Guadagno_Annuo_Stimato DESC, Nome_Impiegato ASC;
```

7.2 Operatori di aggregazione

7.2.1 Quali sono?

- **COUNT:** conta numero di tuple di tabella
- **SUM:** somma i valori o espressioni di attributi
- **MAX:** valore massimo di un attributo di tabella
- **MIN:** valore minimo di un attributo di tabella

- **AVG:** valore medio di un attributo di tabella
- **GROUP BY:** raggruppamento delle tuple in sottogruppi gestiti come estensione delle normali interrogazioni

7.2.2 Come vengono applicati?

1. Si esegue l'interrogazione sulla base di clausole FROM e WHERE
2. Si applica l'operatore aggregato alla tabella risultato dell'interrogazione

7.3 COUNT

7.3.1 Cosa fa?

Restituisce il numero di valori degli attributi in ListaAttr

7.3.2 Sintassi

```
COUNT (< * | [DISTINCT | ALL] ListaAttr >)
```

- **DISTINCT:** numero di valori distinti diversi da NULL di ListaAttr
- **ALL:** numero di valori diversi da NULL di ListaAttr
- **COUNT (*):** conta numero di righe di una tabella

Se non è specificato nulla, ALL è l'opzione di default

7.3.3 Esempio

```
SELECT
    D.Nome AS Nome_Dipartimento,
    D.Citta AS Sede,
    COUNT(I.Matricola) AS Numero_Impiegati
FROM Dip AS D
JOIN Impiegato AS I
    ON D.Nome = I.Dipart
WHERE I.Stipendio > 1500
GROUP BY D.Nome, D.Citta
HAVING COUNT(I.Matricola) > 2
ORDER BY Numero_Impiegati DESC;
```

7.4 SUM

7.4.1 Cosa fa?

Restituisce la somma dei valori di AttrExpr relativamente alle tuple di tabella specificate nella query, opera su attributi numerici

7.4.2 Sintassi

```
SELECT SUM(nome_colonna) AS nome_alias
FROM NomeTabella
[WHERE condizioni_di_filtro];
```

7.4.3 Esempio

```
SELECT SUM(Stipendio) AS Spesa_Totale_Stipendi
FROM Impiegato;
```

7.5 MIN/MAX

7.5.1 Cosa fa?

Restituisce il massimo/minimo dei valori di AttrExpr relativamente alle tuple di tabella specificate nella query, opera su attributi ordinabili

7.5.2 Sintassi

```
SELECT
    MIN(nome_colonna) AS minimo_alias ,
    MAX(nome_colonna) AS massimo_alias
FROM NomeTabella
[WHERE condizioni_di_filtro];
```

7.5.3 Esempio

```
SELECT
    MIN(Stipendio) AS Stipendio_Minimo ,
    MAX(Stipendio) AS Stipendio_Massimo
FROM Impiegato;
```

7.6 AVG

7.6.1 Cosa fa?

Restituisce la media dei valori di AttrExpr relativamente alle tuple di tabella specificate nella query, opera su valori numerici

7.6.2 Sintassi

```
SELECT AVG(nome_colonna) AS nome_alias
FROM NomeTabella
[WHERE condizioni_di_filtro];
```

7.6.3 Esempio

```
SELECT AVG(Stipendio) AS Stipendio_Medio_Aziendale
FROM Impiegato;
```

7.7 Target List

SQL impedisce di includere in una stessa target list funzioni aggregate e espressioni al livello di riga

```
SELECT Cognome, MAX(Stipendio) -- Questa cosa non si puo fare
FROM Impiegato
WHERE Dipart = "Amministrazione"
```

È necessario ricorrere alle subquery, che si vedranno più avanti

7.8 Operatore di aggregamento GROUP BY

7.8.1 Cosa è?

La GROUP BY è una clausola dell'istruzione SELECT utilizzata per raggruppare le righe di una tabella che condividono gli stessi valori in una o più colonne specificate.

7.8.2 Sintassi

```
SELECT colonna_di_raggruppamento, FUNZIONE_AGGREGATA(altra_colonna)
FROM NomeTabella
[WHERE filtri_sulle_singole_righe]
GROUP BY colonna_di_raggruppamento
[HAVING filtri_sui_gruppi_creati]
[ORDER BY criterio_di_ordinamento];
```

7.8.3 Esempio

```
SELECT Dipart, AVG(Stipendio) AS Stipendio_Medio
FROM Impiegato
GROUP BY Dipart;
```

7.9 Condizioni WHERE e HAVING

Specificano entrambi condizioni di selezione:

- **WHERE:** Predicati che operano su tuple individuabili (seleziona di tuple prima di GROUP BY)
- **HAVING:** Predicati che operano su gruppi di tuple creati da GROUP BY (selezioni raggruppamenti)

7.9.1 Sintassi WHERE

```
WHERE attributo OPERATORE valore_o_costante  
[ AND | OR | NOT altra_condizione ];
```

7.9.2 Esempio di uso di WHERE

```
SELECT Cognome, Stipendio  
FROM Impiegato  
WHERE Dipart = 'Amministrazione' AND Stipendio > 1500;
```

7.9.3 Sintassi HAVING

```
GROUP BY colonna_raggruppata  
HAVING FUNZIONE_AGGREGATA(colonna) OPERATORE valore;
```

7.9.4 Esempio di uso di HAVING

```
SELECT Dipart, AVG(Stipendio) AS AvgStipendio  
FROM Impiegato  
GROUP BY Dipart  
HAVING COUNT(*) >= 2; -- Mantiene solo i dipartimenti con almeno 2  
impiegati
```

8. Interrogazioni di tipi insiemistico

8.1 Operatori insiemistici

Permettono di costruire query concatenando due query SQL

- UNION
- INTERSECT
- EXCEPT (Differenza)

8.1.1 Sintassi

```
SelectSQL{< UNION / INTERSECT / EXCEPT > [ALL] SelectSQL}
```

I duplicati vengono eliminati (a meno che si usi ALL)

8.2 UNION

8.2.1 Cosa fa?

Esegue l'unione dei risultati delle 2 query.

L'operatore UNION impone alcune restrizioni sulle interrogazioni su cui opera:

- Le interrogazioni devono restituire lo stesso numero di colonne, e le colonne corrispondenti devono avere lo stesso dominio o domini compatibili
- La corrispondenza si basa sulla posizione delle colonne indipendentemente dal loro nome
- Se si usa la clausola di ORDER BY queste deve essere usata una sola volta alla fine dell'interrogazione e non alla fine di ogni select

8.2.2 Sintassi

```
SelectSQL{ UNION [ALL] SelectSQL}
```

8.2.3 Esempio

```
SELECT CodOrd  
FROM Ordine  
WHERE Importo > 500  
  
UNION  
  
SELECT CodOrd
```

```
FROM Dettaglio  
WHERE Qta > 1000;
```

8.3 INTERSECT

8.3.1 Cosa fa?

Esegue l'intersezione dei risultati delle 2 query.

L'operatore INTERSECT impone alcune restrizioni sulle interrogazioni su cui opera:

- Le interrogazioni devono restituire lo stesso numero di colonne, e le colonne corrispondenti devono avere lo stesso dominio o domini compatibili
- La corrispondenza si basa sulla posizione delle colonne indipendentemente dal loro nome
- Se si usa la clausola di ORDER BY queste deve essere usata una sola volta alla fine dell'interrogazione e non alla fine di ogni select

8.3.2 Sintassi

```
SelectSQL{ INTERSECT [ALL] SelectSQL }
```

8.3.3 Esempio

```
SELECT CodOrd  
FROM Ordine  
WHERE Importo > 500  
  
INTERSECT  
  
SELECT CodOrd  
FROM Dettaglio  
WHERE Qta > 1000;
```

8.4 EXCEPT

8.4.1 Cosa fa?

Esegue la differenza dei risultati delle 2 query.

L'operatore EXCEPT impone alcune restrizioni sulle interrogazioni su cui opera:

- Le interrogazioni devono restituire lo stesso numero di colonne, e le colonne corrispondenti devono avere lo stesso dominio o domini compatibili
- La corrispondenza si basa sulla posizione delle colonne indipendentemente dal loro nome
- Se si usa la clausola di ORDER BY queste deve essere usata una sola volta alla fine dell'interrogazione e non alla fine di ogni select

8.4.2 Sintassi

```
SelectSQL { EXCEPT [ ALL ] SelectSQL }
```

8.4.3 Esempio

```
SELECT Nome  
FROM Impiegato  
  
EXCEPT  
  
SELECT Cognome AS Nome  
FROM Impiegato;
```

9. Interrogazioni Nidificate

9.1 Cosa sono?

Una delle ragioni che rendono SQL un linguaggio potente è la possibilità di esprimere Interrogazioni più complesse in termini di interrogazioni più semplici, tramite il meccanismo delle subqueries

9.2 Come funzionano?

La clauso WHERE di una query può contenere un'altra query.

La subquery viene usata per determinare uno o più valori da usare come valori di confronto in un predicato della query esterna

9.3 Sintassi Subquery

```
SELECT <lista_attributi>
FROM <tabella>
WHERE <valore_scalare> <operatore_confronto> [ANY | ALL]
    (
        SELECT <colonna>
        FROM <tabella_sottoquery>
        [WHERE <condizioni>]
    );
```

9.4 Predicati ANY e ALL

Nella clausola WHERE possono comparire predicati che confrontano un attributo (o un'espressione sugli attributi) con il risultato di una query SQL

- **ANY:** Il predicato è vero se almeno una riga restituita dalla query SelectSQL soddisfa il controllo
- **ALL:** Il predicato è vero se tutte le righe restituite dalla query SelectSQL soddisfano il confronto

9.4.1 Sintassi

```
ScalarValue Operator <ANY | ALL> SelectSQL
```

L'Operato può essere uno tra : =, <>, <, <=, >, >=

9.4.2 Esempio

ANY

```
SELECT Cognome
FROM Impiegato
WHERE Dipart = ANY (SELECT Nome
                    FROM Dip
                    WHERE Citta = 'Milano');
```

ALL

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Stipendio >= ALL (SELECT Stipendio
                       FROM Impiegato);
```

9.5 Quantificatori esistenziali

EXISTS e NOT EXISTS ammettono come parametro le query nidificate.

9.5.1 EXISTS

L'operatore logico EXISTS restituisce il valore booleano true se la subquery restituisce almeno una tupla, false altrimenti

Sintassi

```
EXISTS SelectStar
```

Esempio

```
SELECT A.Citta, A.Nazione
FROM AEROPORTO AS A
WHERE EXISTS (SELECT * FROM VOLO AS V
             WHERE V.CittaPart = A.Citta);
```

9.5.2 NOT EXISTS

L'operatore NOT EXISTS restituisce il valore booleano true se la subquery non restituisce alcuna tupla, false altrimenti

Sintassi

```
NOT EXISTS SelectStar
```

Esempio

```
SELECT O.CodOrd
FROM Ordine AS O
WHERE NOT EXISTS (
    SELECT * FROM Prodotto AS P
    WHERE NOT EXISTS (
        SELECT * FROM Dettaglio AS D
        WHERE D.CodOrd = O.CodOrd
            AND D.CodProd = P.CodProd
    )
);
```

10. Viste

10.1 Cosa sono?

Una vista è una relazione virtuale attraverso è possibile vedere i dati memorizzati nella relazioni reali, chiamate relazioni di base.

10.2 Proprietà fondamentali

- **Natura virtuale:** Una vista non contiene tuple fisiche al suo interno. I suoi dati vengono materializzati dinamicamente eseguendo l'interrogazione (query) che la definisce
- **Utilizzo:** Può essere utilizzata (quasi) nello stesso modo in cui si usa una normale relazione di base
- **Definizione:** Viene definita mediante un'integrazione su una o più relazioni di base, oppure basandosi su altre viste preesistenti. Si crea associando un nome univoco e una lista di attributi al risultato di tale integrazione

10.3 A cosa servono?

Il meccanismo delle viste è particolarmente utile per due scopi principali:

- **Semplificare l'accesso ai dati:** Permette di salvare query complesse sotto un unico nome, rendendo le successive interrogazioni molto più semplici.
- **Garantire la privacy dei dati:** Consente di mostrare a determinati utenti solo una parte specifica dei dati (un sottoinsieme di righe o colonne), nascondendo il resto della relazione di base

10.4 Sintassi

```
CREATE VIEW NomeVista [(ListaAttributi)] AS query
```

- **NomeVista:** È il nome della vista che viene create, tale nome deve essere unico rispetto a tutti i nomi di relazioni e di viste definite dallo stesso utente che la definisce
- **query:** È l'interrogazione di definizione della vista. Una vista ha lo stesso numero di colonne pari alle colonne (di base o virtuali) specificate nella clausola di proiezione di query
- **ListaAttributi:** È una lista di nomi da assegnare alle colonne della vista, tale specifica non è obbligatoria, tranne nel caso in cui l'interrogazione contenga nella clausola di proiezioni funzioni di gruppo e/o espressioni

10.5 Esempio creazione vista

```
CREATE VIEW Personale AS  
SELECT Nome, Mansione, Pnome  
FROM Impiegati, Progetti  
WHERE Impiegati.ProgNo = Progetti.ProgNo;
```

11. La Progettazione Logica in Pratica

L'obiettivo principale della progettazione logica è "tradurre" lo schema concettuale (il diagramma Entità-Relazione) in uno schema logico relazionale (fatto di tabelle) che rappresenti gli stessi dati in maniera corretta ed efficiente. L'ingresso di questa fase è costituito dallo schema concettuale e dalle informazioni sul carico applicativo, mentre l'uscita è lo schema logico con la relativa documentazione. Non si tratta però di una pura e semplice traduzione diretta, perché alcuni aspetti dello schema ER (come le generalizzazioni) non sono direttamente rappresentabili nel modello relazionale.

La progettazione si divide in due grandi fasi sequenziali: la ristrutturazione dello schema e la traduzione vera e propria.

11.1 Ristrutturazione dello Schema E-R

Prima di tradurre, devi "pulire" e preparare il tuo diagramma E-R eliminando i costrutti non supportati dal modello logico di destinazione. Le prestazioni ottimali non sono valutabili con precisione solo guardando lo schema concettuale, ma dipendono dal volume dei dati e dalle caratteristiche delle operazioni.

11.1.1 Analisi delle Ridondanze

- Una ridondanza è un'informazione derivabile da altri dati presenti nel database.
- In questa prima fase devi analizzare le ridondanze e decidere se eliminarle o mantenerle.
- Mantenerle ha il vantaggio di semplificare e velocizzare alcune interrogazioni, ma penalizza le prestazioni occupando più spazio in memoria e appesantendo le operazioni di aggiornamento.
- Come regola generale, gli accessi ai dati si riducono separando gli attributi di un concetto che vengono acceduti separatamente e, viceversa, raggruppando gli attributi di concetti diversi che vengono acceduti insieme.

11.1.2 Eliminazione degli Attributi Multivalore

- Il modello relazionale non ammette che un singolo campo possieda più valori contemporaneamente.
- Tali attributi multivalore vanno obbligatoriamente eliminati e trasformati (generalmente in nuove entità collegate).

11.1.3 Eliminazione delle Gerarchie (Generalizzazioni)

- Il modello relazionale non può rappresentare direttamente l'ereditarietà tra un'entità "padre" e le sue entità "figlie".
- Per risolvere il problema, si eliminano le gerarchie sostituendole con entità e relazioni standard.

- Esistono tre tecniche fondamentali per questa operazione:
 1. **Accorpamento delle figlie nel genitore:** Si unisce tutto nell'entità padre, tecnica considerata sempre applicabile. Si aggiunge un attributo (es. *Tipo*) per distinguere le istanze, accettando però che i campi specifici delle figlie assumano valori nulli quando non pertinenti. Questa scelta conviene se gli accessi al padre e alle figlie sono contestuali. È invece sconsigliata se le figlie portano in dote un elevato numero di relazioni e/o attributi.
 2. **Accorpamento del genitore nelle figlie:** Si elimina l'entità padre e si spostano tutti i suoi attributi in ciascuna tabella figlia. Conviene se gli accessi alle figlie sono del tutto distinti e **solo se** la generalizzazione è totale. È una tecnica fortemente sconsigliata in presenza di coperture parziali o sovrapposte.
 3. **Sostituzione con relazioni:** Si mantengono sia il padre che le figlie separate, collegandole con relazioni uno-a-uno. È una soluzione molto generale e conviene quando gli accessi alle figlie sono separati dagli accessi al padre e quando la generalizzazione non è totale (evitando così di introdurre valori nulli). Ha il "contro" di incrementare il numero degli accessi necessari per ricostruire l'informazione di partenza e gestire i vincoli.

11.2 Traduzione verso il Modello Relazionale

Una volta che il tuo schema è ristrutturato ed è stato "appiattito", puoi applicare le regole di derivazione logica per generare le tabelle finali. L'idea di base è che le entità diventano relazioni sugli stessi attributi, mentre le associazioni si traducono usando gli identificatori (le chiavi) delle entità coinvolte.

11.2.1 Traduzione delle Entità

- Il nome dell'entità diventa direttamente il nome della tua tabella.
- Tra parentesi, elenca tutti gli attributi dell'entità.
- L'identificatore primario dell'entità diviene la Chiave Primaria (Primary Key) della tabella e va sottolineato (`\underline{Chiave}`).
- Nel caso di entità con identificatore esterno (dipendenza da un'altra entità), la chiave della tabella includerà a cascata anche la chiave dell'entità identificante.

11.2.2 Traduzione delle Relazioni Multi-a-Multi (N:N)

- Ogni associazione N:N si trasforma sempre in una nuova tabella a sé stante.
- Questa tabella conterrà come chiavi esterne gli identificatori delle due entità che sta collegando.
- Insieme, questi identificatori formeranno la superchiave (Chiave Primaria) della nuova tabella.
- Eventuali attributi propri dell'associazione (ad esempio una data di inizio) devono essere inseriti all'interno di questa nuova tabella.

- È possibile scegliere nomi più espressivi per gli attributi che compongono la chiave per aumentarne la chiarezza.

11.2.3 Traduzione delle Relazioni Uno-a-Molti (1:N)

- Per le associazioni 1:N non si crea quasi mai una nuova tabella dedicata.
- La regola impone di prendere la chiave primaria dell'entità che si trova dal lato "molti" (quella con cardinalità massima N) e inserirla come chiave esterna (Foreign Key) dentro la tabella dell'entità che si trova dal lato "uno" (quella con cardinalità massima 1).
- Eventuali attributi propri dell'associazione vanno anch'essi spostati sempre nella tabella dell'entità dal lato "uno".

11.2.4 Traduzione delle Relazioni Uno-a-Uno (1:1)

- Similmente al caso precedente, non si creano nuove tabelle.
- Si inserisce la chiave primaria di un'entità come chiave esterna all'interno della tabella dell'altra entità collegata.
- Come prassi logica, si preferisce inserire la chiave esterna nell'entità che ha una partecipazione obbligatoria all'associazione (ovvero cardinalità minima pari a 1) per evitare di generare e gestire campi vuoti (NULL) nel database.
- Se entrambe le entità hanno partecipazione obbligatoria, hai la libertà di scegliere in quale delle due inserire la chiave esterna. Se entrambe hanno partecipazione opzionale, si può tollerare un attributo nullo oppure introdurre una tabella aggiuntiva dedicata all'associazione per mantenere uno schema più pulito.

12. Esercizi MR

12.1 Totale Giugno 2026

Si desidera progettare una base di dati per gestire le attività di una società di spedizioni internazionali che ha diverse filiali nel mondo. Il sistema deve riflettere le seguenti specifiche: Ogni Filiale della società è identificata da un codice univoco, ha un nome (es. "Sede Milano"), un indirizzo completo (via, civico, città) e un responsabile della filiale che è unico per ogni sede. Ogni filiale gestisce diversi Magazzini; ogni magazzino è identificato da un numero progressivo all'interno della singola filiale, ha una metratura e una tipologia (es. "Refrigerato", "Merci Pericolose"). I Pacchi da spedire sono identificati da un codice di tracciamento (tracking) unico, hanno un peso, un volume, una descrizione del contenuto e l'anno di produzione dell'imballaggio. Ogni pacco è attualmente stoccato in uno specifico magazzino di una filiale e si vuole tenere traccia di questa posizione. Per ogni Modalità di Spedizione (es. "Aerea", "Navale", "Su Gomma"), si definisce un costo base di gestione in una specifica data di aggiornamento del listino logistico. Si registrano le singole Spedizioni effettuate. Ogni spedizione avviene presso una specifica filiale, in una certa data e ora, indicando la modalità di spedizione utilizzata e il corriere (dipendente) che ha preso in carico il carico, se già assegnato. Sia i corrieri che i responsabili sono Dipendenti della società. Per ogni dipendente, identificato dal codice fiscale, si vuole memorizzare: nome, cognome, data di nascita, data di assunzione, Email e Telefono.

- FILIALE(Codice, Nome, Via, Civico, Citta, Responsabile)
- MAGAZZINO(CodiceFiliale, NumeroMagazzino, Metratura, Tipologia)
- PACCO(CodiceTracking, Peso, Volume, Contenuto, AnnoImballaggio, CodiceFiliale, NumeroMagazzino)
- LISTINO_LOGISTICO(CodiceFiliale, ModalitaSpedizione, CostoBase)
- SPEDIZIONE(IdSpedizione, CodiceFiliale, ModalitaSpedizione, Data, Ora, Corriere)
- DIPENDENTE(CF, Nome, Cognome, DataNascita, DataAssunzione, Email, Telefono)

12.1.1 Chiavi primarie e vincoli di integrità referenziale

- Chiavi primarie :
 - FILIALE(Codice, Nome, Via, Civico, Citta, Responsabile)
 - MAGAZZINO(CodiceFiliale, NumeroMagazzino, Metratura, Tipologia)
 - PACCO(CodiceTracking, Peso, Volume, Contenuto, AnnoImballaggio, CodiceFiliale, NumeroMagazzino)
 - LISTINO_LOGISTICO(CodiceFiliale, ModalitaSpedizione, CostoBase)
 - SPEDIZIONE(IdSpedizione, CodiceFiliale, ModalitaSpedizione, Data, Ora, Corriere)
 - DIPENDENTE(CF, Nome, Cognome, DataNascita, DataAssunzione, Email, Telefono)
- Vincoli di integrità referenziale:

- **FILIALE**
 - * Responsabile → DIPENDENTE(CF)
- **MAGAZZINO**
 - * CodiceFiliale → FILIALE(Codice)
- **PACCO**
 - * (CodiceFiliale, NumeroMagazzino) → MAGAZZINO(CodiceFiliale, NumeroMagazzino)
- **LISTINO_LOGISTICO**
 - * CodiceFiliale → FILIALE(Codice)
- **SPEDIZIONE**
 - * CodiceFiliale → FILIALE(Codice)
 - * (CodiceFiliale, ModalitaSpedizione) → LISTINO_LOGISTICO(CodiceFiliale, ModalitaSpedizione)
 - * Corriere → DIPENDENTE(CF)

12.1.2 Due vincoli intrarelazionali di dominio, uno di tupla.

Vincoli intrarelazionali

1. CF CHAR(16)
2. Peso INT > 0

Vincolo di tupla

- DataNascita < DataAssunzione

12.1.3 Chiave alternativa per la relazione FILIALE e DIPENDENTE

- DIPENDENTE(CF, Nome, Cognome, DataNascita, DataAssunzione, Email, Telefono)
- FILIALE(Codice, Nome, Via, Civico, Citta, Responsabile)

12.1.4 Superchiave per la relazione PACCO

(CodiceTracking, Peso, Volume, Contenuto, AnnoImballaggio, CodiceFiliale, NumeroMagazzino)

13. Esercizi AR

13.1 Totale Giugno 26

13.1.1 Esercizio 1

Trovare i nick e le nazioni dei giocatori che hanno partecipato ad almeno un torneo disputato a Milano con montepremi di almeno 50.

$$\pi_{Nick, Nazione} (GIOCATORE \bowtie (\sigma_{Città='Milano' \wedge Montepremi \geq 50}(TORNEO)) \bowtie PARTECIPAZIONE)$$

13.1.2 Esercizio 2

Trovare i nick dei giocatori italiani che non hanno mai partecipato a un torneo nel 2026.

$$(\pi_{Nick} (\sigma_{Nazione='Italia'}(GIOCATORE))) - (\pi_{Nick} (PARTECIPAZIONE \bowtie (\sigma_{Anno=2026}(TORNEO))))$$

13.2 Totale Giugno 2017

Date le relazioni:

- **NEGOZIO**(codiceN, nomeN, catena)
- **PRODOTTO**(codiceP, nomeP, TipoP, Prezzo, codiceN)

13.2.1 Es 1

Trovare il nome dei negozi che hanno almeno un prodotto di tipo "alimentare"

$$\pi_{\text{nomeN}}(\sigma_{\text{TipoP}=\text{Alimentare}}(\text{Prodotto} \bowtie \text{Negozio}))$$

13.2.2 Es 2

Trovare il codice dei negozi che non hanno prodotti di tipo "alimentare"

$$\pi_{\text{codiceN}}(\text{NEGOZIO}) \setminus \pi_{\text{codiceN}}(\sigma_{\text{TipoP}=\text{'alimentare'}}(\text{PRODOTTO}))$$

13.2.3 Es 3

Trovare il nome dei negozi che hanno almeno sia prodotti di tipo "alimentare" che "casa".

$$\pi_{\text{nomeN}}(\sigma_{\text{TipoP}=\text{'alimentare'}}(\text{NEGOZIO} \bowtie \text{PRODOTTO})) \cap \pi_{\text{nomeN}}(\sigma_{\text{TipoP}=\text{'casa'}}(\text{NEGOZIO} \bowtie \text{PRODOTTO}))$$

13.3 Totale Settembre 2015

Date le relazioni,

- **STUDENTE** (matr, nome, cognome)
- **ESAME** (matr, cod-corso, voto)
- **CORSO**(cod-corso, nome-corso, docente)

13.3.1 Es 1

Trovare nome, cognome e matricola degli studenti che hanno registrato il voto dell'esame di "basi di dati"

$$\pi_{\text{nome, cognome, matr}}(\text{Studente} \bowtie \text{Esame} \bowtie \sigma_{\text{nome-corso}=\text{'Basi di Dati'}}(\text{Corso}))$$

13.3.2 Es 2

Trovare nome, cognome e matricola degli studenti che non hanno superato alcun esame (nota: solo gli esami con voto positivo vengono registrati nella relazione esame)

$$\pi_{\text{nome, cognome, matr}}(\text{STUDENTE}) \setminus \pi_{\text{nome, cognome, matr}}(\text{STUDENTE} \bowtie \text{ESAME})$$

13.3.3 Es 3

Trovare nome e cognome degli studenti omonimi

$$\pi_{S1.\text{nome}, S1.\text{cognome}}(\sigma_{S1.\text{nome}=S2.\text{nome} \wedge S1.\text{cognome}=S2.\text{cognome} \wedge S1.\text{matr} \neq S2.\text{matr}}(\rho_{S1}(\text{STUDENTE}) \times \rho_{S2}(\text{STUDENTE})))$$

13.4 Slide AR1

13.4.1 Es 1

- IMPIEGATO(CodImp, Cognome, Età, Stipendio, CodDip)
- DIPARTIMENTO(CodDip, NomeDip, Telefono, Direttore)

Elencare Cognome degli impiegati con Stipendio > 3000€ insieme al NomeDip del loro dipartimento.

$$\Pi_{\text{Cognome Imp, Nome Dip}} (\sigma_{\text{Stipendio} > 3000}(\text{Impiegato}) \bowtie \text{Dipartimento})$$

13.4.2 Es 2

- IMPIEGATO(CodImp, Cognome, CodDip)
- PARTECIPAZIONE(CodImp, CodProg)
- PROGETTO(CodProg, NomeProg, Budget)

Elencare Cognome e NomeProg per tutti gli impiegati che partecipano a progetti con budget > 150000€

$$\Pi_{\text{Cognome, NomeProg}} (\text{IMPIEGATO} \bowtie \text{PARTECIPAZIONE} \bowtie \sigma_{\text{Budget} > 150000}(\text{PROGETTO}))$$

13.4.3 Es 3

- IMPIEGATO(CodImp, Cognome, Età, CodDip)

Trovare le coppie Senior (età > 50) e Junior (età < 30) di impiegati dello stesso dipartimento

$$\sigma_{I1.Et'a > 50 \wedge I2.Et'a < 30 \wedge I1.CodDip = I2.CodDip} (\rho_{I1}(\text{IMPIEGATO}) \times \rho_{I2}(\text{IMPIEGATO}))$$

13.4.4 Es 4

- IMPIEGATO(CodImp, Cognome, Età, CodDip)

Trovare gli impiegati Junior (età < 30) e quelli Senior (età > 50). Costruire tre insiemi di CodImp:

- Tutti gli impiegati che non hanno età tra 30 e 50 compresi.
- Gli impiegati solo Junior (cioè, quelli con età < 30)
- Impiegati sia Junior che Senior (cioè, sia quelli con età < 30 sia quelli con età > 50)

A

$$\Pi_{\text{CodImp}} (\sigma_{Et'a < 30 \vee Et'a > 50}(\text{IMPIEGATO}))$$

B

$$\Pi_{\text{CodImp}}(\sigma_{\text{Et'a} < 30}(\text{IMPIEGATO}))$$

C

$$\Pi_{\text{CodImp}}(\sigma_{\text{Et'a} < 30}(\text{IMPIEGATO})) \cup \Pi_{\text{CodImp}}(\sigma_{\text{Et'a} > 50}(\text{IMPIEGATO}))$$

13.4.5 Es 5

- IMPIEGATO(CodImp, CodDip, Stipendio)
- DIPARTIMENTO(CodDip, NomeDip, Telefono)

Dato il piano $\pi_{\text{NomeDip}}(\pi_{\text{CodImp, CodDip, NomeDip}}(\text{IMPIEGATO} \bowtie \text{DIPARTIMENTO}))$, si vuole un piano ottimizzato

13.5 Slide AR2

13.5.1 Es 1

- ESAME(Matricola, CodCorso, Voto)
- CORSO_OBBLIG(CodCorso) – corsi obbligatori

Elencare le matricole degli studenti che hanno superato ($\text{Voto} \geq 18$) **tutti** i corsi obbligatori

$$\Pi_{\text{Matricola, CodCorso}}(\sigma_{\text{Voto} \geq 18}(\text{ESAME})) \div \text{CORSO_OBBLIG}$$

13.5.2 Es 2

- PRODOTTO(PID, Categoria)
- VENDITA(NumOrd, PID, Quantità, PrezzoUnit)

Trovare il **fatturato totale** ($\text{Quantità} * \text{PrezzoUnit}$) per categoria

$$\gamma_{\text{Categoria, SUM(Quantità} \times \text{PrezzoUnit)}}(\text{PRODOTTO} \bowtie \text{VENDITA})$$

13.5.3 Es 3

- CORSO(CodCorso, Titolo)
- VALUTAZIONE(ID, CodCorso, Stelle) – le stelle vanno da 1 a 5

Mostrare tutti i corsi con la **media delle stelle**; se nessuna valutazione, la media rimane NULL

$$\gamma_{\text{CodCorso:AVG(Stelle)}}(\text{CORSO} \bowtie_{\text{LEFT CORSO.CodCorso=VALUTAZIONE.CodCorso}} \text{VALUTAZIONE})$$

13.5.4 Es 4

- RICERCATORE(RID, Nome)
- PROGETTO(PID, Titolo, RID) – responsabile opzionale

Elencare Titolo progetto e Nome responsabile, mostrando NULL se il responsabile non è registrato

$$\Pi_{\text{Titolo, Nome}}(\text{RICERCATORE} \bowtie_{\text{RIGHT RICERCATORE.RID=PROGETTO.RID}} \text{PROGETTO})$$

13.5.5 Es 5

- ARTISTA(AID, Nome)
- ALBUM(AID, Titolo)

Produrre elenco combinato di Artista–Titolo, includendo artisti senza album e album senza artista registrato

14. Esercizi SQL

14.1 Totale Giugno 2026

Si consideri il seguente schema relazionale:

DIPENDENTE(ID_impiegato, CF, cognome, nome, data_assunzione)

MANSIONE(codice, titolo, stipendio_base)

PREMIO_PRODUZIONE(dipendente, mansione, data_assegnazione, importo_premio)

14.1.1 Query 1

Il titolo e lo stipendio base delle mansioni ordinate prima in ordine decrescente per stipendio base e poi per ordine alfabetico.

```
CREATE VIEW AS info
SELECT titolo, stipendio_base
FROM MANSIONE;

(
    SELECT * FROM info
    ORDER BY stipendi_base DESC
)
UNION ALL
(
    SELECT * FROM info
    ORDER BY titolo ASC
)
```

14.1.2 Query 2

Il codice e il titolo delle mansioni che valgono il maggior stipendio base inserito a sistema.

```
SELECT M.codice, M.titolo
FROM MANSIONE AS M
WHERE M.stipendio_base = (
    SELECT MAX(stipendio_base)
    FROM MANSIONE
);
```

14.1.3 Query 3

Il codice, il titolo della mansione e la media dell'importo del premio per ciascuna mansione presente nella tabella dei premi

```
SELECT M.codice, M.titolo, AVG(P.importo_premio) AS media_premio
FROM MANSIONE AS M
JOIN PREMIO_PRODUZIONE AS P ON M.codice = P.mansione
```

```
GROUP BY M.codice, M.titolo;
```

14.1.4 Query 4

L'ID impiegato, il nome e il cognome del dipendente che ha ricevuto il premio di produzione più basso in assoluto all'interno della mansione avente codice 'MAN01'.

```
SELECT D.ID_impiegato, D.nome, D.cognome
FROM DIPENDENTE AS D JOIN PREMIO_PRODUZIONE AS P ON D.Id_impiegato = P.
    dipendente
WHERE P.importo_premio = (
    SELECT MIN(importo_premio)
    FROM PREMIO_PRODUZIONE
    WHERE mansione = 'MAN01'
)
```

14.2 Totale Giugno 2015

Sia dato il seguente schema che descrive il database di un negozio di elettronica di consumo

- prodotto(codice, nome, marca, prezzo)
- scontrino (progressivo, data, ora, articolo, tesseraCliente)
- cliente (Tessera, nome, cognome, cittaResidenza, puntiFedelta)

14.2.1 Query 1

Trovare il nome del prodotto più venduto tra le 11 e le 12 di qualunque giorno

```
SELECT P.nome
FROM Prodotto AS P
JOIN SCONTRINO AS S ON S.articolo = P.codice
GROUP BY P.codice, P.nome
HAVING COUNT(*) >= ALL (
    SELECT COUNT
    FROM SCONTRINO AS S2
    JOIN SCONTRINO AS S2 ON S2.articolo = P.codice
    WHERE S2.ora BETWEEN '11:00' AND '12:00'
    GROUP BY S2.articolo
);
```

14.2.2 Query 2

Trovare il nome e il cognome dei clienti che pur avendo comprato un prodotto della "Apple" non hanno mai comprato un prodotto della "Samsung"

```
SELECT C.nome, C.cognome
FROM CLIENTE AS C
WHERE EXISTS (
    SELECT P.marca
    FROM PRODOTTO AS P JOIN SCONTRINO AS S ON P.codice = S.articolo
    WHERE P.marca = "Apple" AND S.tesseraCliente = Cliente.tesseraCliente
)
AND
NOT EXISTS (
    SELECT P.marca
    FROM PRODOTTO AS P JOIN SCONTRINO AS S ON P.codice = S.articolo
    WHERE P.marca = "Samsung" AND S.tesseraCliente = Cliente.
        tesseraCliente
)
```

14.2.3 Query 3

Trovare il nome e cognome dei clienti di Milano che hanno comprato contemporaneamente un prodotto "Philips" e un prodotto "Dell"

```
SELECT C.nome, C.cognome
FROM CLIENTI AS C JOIN SCONTRINO AS S ON C.Tessera = S.tesseraCliente
WHERE C.cittaResidenza = 'Milano' AND EXISTS (
    SELECT P.marca
    FROM PRODOTTO AS P JOIN SCONTRINO AS S1 ON P.codice = S1.articolo
    WHERE P.marca = 'Philips' AND S1.tesseraCliente = C.Tessera
)
AND EXISTS (
    SELECT P.marca
    FROM PRODOTTO AS P JOIN SCONTRINO AS S1 ON P.codice = S1.articolo
    WHERE P.marca = 'Dell' AND S1.tesseraCliente = C.Tessera
)
```

14.3 2° Parziale 2016

- STUDENTE (matr, nome, cognome, LuogoNascita)
- ESAME (matr, codCorso, voto, data)
- CORSO(codCorso, nomeCorso, docente)

14.3.1 Query 1

Trovare la città in cui sono nati gli studenti che complessivamente hanno superato più esami rispetto agli studenti nati in altre città

```
SELECT S.LuogoNascita
FROM STUDENTE AS S JOIN ESAME AS E ON S.matr = E.matr
GROUP BY S.LuogoNascita HAVING COUNT(*) >= ALL (
    SELECT COUNT(*)
    FROM STUDENTE AS S1 JOIN ESAME AS E1 ON S1.matr = E1.matr
    WHERE E1.voto >= 18
    GROUP BY S1.LuogoNascita
)
```

14.3.2 Query 2

Trovare nome cognome e matricola, degli studenti che hanno superato l'esame di analisi con voto superiore al voto ottenuto per l'esame di fisica

```
SELECT S.nome, S.cognome, S.matr
FROM STUDENTE AS S JOIN ESAME AS E1 ON S.matr = E1.matr
JOIN CORSO AS C1 ON E1.codCorso = C1.codCorso
JOIN ESAME AS E2 ON S.matr = E2.matr
JOIN CORSO AS C2 ON E2.codCorso = C2.codCorso
WHERE C1.nomeCorso = 'Analisi' AND C2.nomeCorso = 'Fisica'
AND E1.voto > E2.voto
```

14.3.3 Query 3

Trovare il nome dei corsi di cui studenti omonimi hanno superato l'esame

```
SELECT C.nomeCorso
FROM CORSO AS C
JOIN ESAME AS E1 ON C.codCorso = E1.codCorso
JOIN ESAME AS E2 ON C.codCorso = E2.codCorso
JOIN STUDENTE AS S1 ON E1.matr = S1.matr
JOIN STUDENTE AS S2 ON E2.matr = S2.matr
WHERE S1.matr <> S2.matr
AND S1.nome = S2.nome AND S1.cognome = S2.cognome
AND E1.codCorso = E2.codCorso
AND E1.voto >= 18 AND E2.voto >= 18
```

14.4 Totale Gennaio 2017

La seguente base di dati descrive l'attività di una società di noleggio di autovetture che ha diverse sedi in alcune città italiane.

- **SEDE**(NomeSede, indirizzo, città, telefono, direttore)
- **AUTOVETTURA**(targa, modello, AnnoImmatricolazione, numeroposti, colore)
- **CLIENTE**(CF, nome, cognome, telefono, città)
- **CONTRATTO**(auto, cliente, DataNoleggio, SedeNoleggio, DataConsegna, SedeConsegna)

14.4.1 Query 1

Trovare il numero di autovetture noleggiate in ogni città, da clienti con residenza in una città diversa dalla città in cui avviene il noleggio.

```
SELECT S.città ,COUNT(*) AS numero_noleggi
FROM CONTRATTO AS C
JOIN CLIENTE AS CE ON C.cliente = C.CF
JOIN SEDE AS S ON S.NomeSede = C.SedeNoleggio
WHERE S.città <> CE.città
GROUP BY S.città
```

14.4.2 Query 2

Trovare i clienti (indicando CF, nome e cognome) che hanno noleggiato almeno due macchine diverse nella stessa sede di noleggio.

```
SELECT C.CF, C.nome, C.cognome
FROM CLIENTE AS C
JOIN CONTRATTO AS C01 ON C.CF = C01.cliente
JOIN CONTRATTO AS C02 ON C.CF = C02.cliente
WHERE C01.auto <> C02.auto
AND C01.SedeNoleggio = C02.SedeNoleggio
```

14.4.3 Query 3

Trovare la sede di noleggio, indicandone anche la città, che ha realizzato il maggior numero di contratti della durata di un giorno (cioè in cui la macchina sia stata riconsegnata lo stesso giorno in cui è stata noleggiata).

```
SELECT S.NomeSede, S.città
FROM SEDE AS S
JOIN CONTRATTO AS C ON S.NomeSede = C.SedeNoleggio
GROUP BY S.Nome, S.città
HAVING COUNT(*) >= ALL (
    SELECT COUNT(*)p
    FROM CONTRATTO
    WHERE DataConsegna = SedeConsegna
    GROUP BY SedeNoleggio
)
```

14.5 Totale Luglio 2017

La seguente base di dati descrive l'archivio di una società di charter che organizza e vende vacanze in barca. La società dispone di una flotta di barche ed organizza viaggi con diverse destinazioni.

- **VIAGGIO** (idCliente, dataPartenza, nomeBarca, Skipper, portoPartenza, portoArrivo, costo)
- **CLIENTE** (idCliente, nome, cognome, nazione)
- **FLOTTA** (NomeBarca, modello, lunghezza, posti, annoImmatricolazione)
- **PORTO** (Nome, nazione, posti, telefono, canaleRadio)

14.5.1 Query 1

Trovare la nazione che è stata coinvolta nella maggior parte dei viaggi (come porto di arrivo o come porto di partenza).

```
SELECT P.nazione
FROM PORTO AS P
JOIN VIAGGIO AS V ON V.portoPartenza = P.Nome OR V.portoArrivo = P.Nome
GROUP BY P.nazione
HAVING COUNT(*) >= ALL (
    SELECT COUNT(*)
    FROM PORTO AS P2
    JOIN VIAGGIO AS V2 ON V2.portoPartenza = p2.Nome OR V2.portoArrivo =
        p2.Nome
    GROUP BY P2.nazione
)
```

14.5.2 Query 2

Trovare il nome delle barche che non sono mai state affittate.

```
SELECT F.NomeBarca
FROM FLOTTA AS F
WHERE F.NomeBarca NOT IN (
    SELECT V.nomeBarca
    FROM VIAGGIO
)
```

14.5.3 Query 3

Trovare per ogni porto il costo medio dei viaggi da lì partiti che non abbiano imbarcato clienti della nazionalità del porto stesso.

```
SELECT P.nome, AVG(V.costo)
FROM PORTO AS P, VIAGGIO AS V
JOIN CLIENTE AS C ON V.idCliente = C.idCliente
WHERE C.nazione <> P.nazione
GROUP BY P.Nome
```

14.6 Esercizio 1

Data la seguente base di dati:

- **STUDENTE**(matricola, CF, cognome, nome, data_nascita)
- **INSEGNAMENTO**(codice, nome, crediti)
- **ESAME**(studente, insegnamento, data, voto, lode)

14.6.1 Query 1

Il codice e il nome degli insegnamenti che valgono il maggior numero di crediti

```
SELECT I.codice, I.nome
FROM INSEGNAMENTO AS I
WHERE I.crediti = ALL (
    SELECT I2.crediti
    FROM INSEGNAMENTO AS I2
)
```

14.6.2 Query 2

Matricola e media voto degli studenti nati nel 2002

```
SELECT S.matricola, AVG(E.voto)
FROM STUDENTE AS S, ESAME AS E
JOIN S ON E.studente = S.matricola
WHERE S.data_nascita = '2002'
GROUP BY S.matricola
```

14.6.3 Query 3

La matricola, il nome, il cognome dello studente che ha preso il massimo voto nell'esame dell'insegnamento di codice 'BDD01'

```
SELECT S.matricola, S.nome, S.cognome
FROM STUDENTE AS S
JOIN ESAME AS E ON S.matricola = E.studente
JOIN INSEGNAMENTO AS I ON I.codice = E.insegnamento
WHERE E.insegnamento = 'BDD01' AND E.voto = (
    SELECT MAX(VOTO)
    FROM ESAME
    WHERE E.insegnamento = 'BDD01'
)
```

14.7 Esercizio 2

Data la seguente base di dati

- **Aeroporto**(Città, Nazione, NumPiste)
- **Volo**(IdVolo, GiornoSett, CittàPart, OraPart, CittàArr, OraArr, Aereo)
- **Aereo**(TipoAereo, NumPasseggeri, QtaMerci)

14.7.1 Query 1

Selezionare tutti gli ID dei voli che utilizzano l’AIRBUS 400

```
SELECT V.IdVolo
FROM VOLO AS V
WHERE V.Aereo = 'AIRBUS_400'
```

14.7.2 Query 2

Selezionare gli aeroporti che hanno il maggior numero di piste

```
SELECT Citta
FROM Aeroporto AS A
WHERE A.NumPiste = (
    SELECT MAX(A1.NumPiste)
    FROM Aeroporto AS A1
)
```

14.7.3 Query 3

Selezionare le tipologie di aerei che hanno viaggiato in italia

```
SELECT DISTINCT V.Aereo
FROM Volo AS V
JOIN Aeroporto AS AR1 ON V.CittàPart = AR1.Città
JOIN Aeroporto AS AR2 ON V.CittàArr = AR2.Città
Where AR1.Nazione = 'Italia' OR AR2.Nazione = 'Italia'
```

14.7.4 Query 4

Selezionare le tipologie di aerei che hanno viaggiato maggiormente

14.7.5 Query 5

Selezionare le tipologie di aerei che hanno effettuato più di 100 voli

```
SELECT V.Aereo  
FROM VOLO AS V  
GROUP BY V.Aereo  
HAVING COUNT(*) > 100
```

14.8 Esercizio 3

Data la seguente base di dati che descrive un campionato invernale di sci maschile

- **LUOGOGARA**(nome, altitudine, nazione)
- **GARA** (specialità, luogo, dataGara, lunghezza, dislivello, premiovincitore)
- **POSIZIONEGARA** (idatleta, dataGara, numeroPettorina, specialità, luogo, posizionearrivo)
- **ATLETA** (idatleta, nome, cognome, nazione, datadinascita)

14.8.1 Query 1

Trovare per ogni nazione l'atleta che ha vinto il maggior numero di gare

```
SELECT A.idatleta
FROM ATLETA AS A
JOIN POSIZIONEGARA AS P ON P.idatleta = A.idatleta
WHERE P.posizionearrivo = 1
GROUP BY A.nazione, A.idatleta
HAVING COUNT(*) >= ALL (
    SELECT COUNT(*)
    FROM ATLETA AS A2 JOIN POSIZIONEGARA AS P2 ON A2.idatleta = P2.
        idatleta
    WHERE posizionearrivo = 1 AND A.nazione = A2.nazione
    GROUP BY A2.idatleta
)
```

14.8.2 Query 2

Trovare le nazioni che hanno almeno un'atleta che ha vinto almeno una gara in patria

```
SELECT DISTINCT A.nazione
FROM ATLETA AS A
JOIN POSIZIONEGARA AS P ON A.idatleta = P.idatleta
JOIN LUOGOGARA AS L ON L.nome = P.luogo
WHERE P.posizionearrivo = 1 AND L.nazione = A.nazione
```

14.9 Esercizio 4

- **CLIENTE**(CodiceFiscale, Nome, Cognome, Nazionalità, Telefono, e-mail)
- **STANZA**(CodiceStanza, Albergo, Tipo, MetriQuadri, Locali, Piano)
- **SOGGIORNO**(CodiceStanza, Albergo, DataInizio, DataFine, Agente, CFCliente, CostoGiornaliero)
- **ALBERGO**(CodiceAlbergo, NumeroDiStelle, Indirizzo, Città, Stato)
- **AGENTE**(CodiceAgente, Nome, Cognome, DataDiNascita)

14.9.1 Query 1

Trovare, specificando nome, cognome e codice fiscale, i clienti omonimi che abbiano soggiornato in una qualunque stanza di albergo nella medesima città

```
SELECT C1.Nome , C1.Cognome , C1.CodiceFiscale
FROM CLIENTE AS C1
JOIN SOGGIORNO AS S1 ON C1.CodiceFiscale = S1.CFCLIENTE
JOIN ALBERGO AS AL1 ON S1.Albergo = AL1.CodiceAlbergo
WHERE EXISTS (
    SELECT *
    FROM CLIENTE AS C2
    JOIN SOGGIORNO AS S2 ON C2.CodiceFiscale = S2.CFCLIENTE
    JOIN ALBERGO AS A2 ON S2.Albergo = A2.CodiceAlbergo
    WHERE C1.Nome = C2.Nome AND C1.Cognome = C2.Cognome AND C1.
        CodiceFiscale <> C2.CodiceFiscale AND AL1.Città = A2.Città
)
```

14.9.2 Query 2

Trovare l'agente (codice, nome, cognome) che ha stipulato più soggiorni per l'anno 2009 in alberghi con più di 3 stelle

```
SELECT AG.CodiceAgente , AG.Nome , AG.Cognome
FROM AGENTE AS AG
JOIN SOGGIORNO AS S ON S.Agente = AG.CodiceAgente
JOIN ALBERGO AS AL ON AG.Albergo = AL.CodiceAlbergo
WHERE S.DataInizio >= 1-1-2009 AND S.DataInizio < 1-1-2010 AND AL.
    NumeroDiStelle > 3
GROUP BY AG.CodiceAgente , AG.Nome , AG.Cognome
HAVING COUNT(*) >= ALL
```

14.10 Esercizio 5

- PALAZZETTO(Nome, Città, Capienza)
- INCONTRO(NomePalazzetto, Data, Ora, Squadra1, Squadra2)
- NAZIONALE(Nazione, Continente, Livello).

14.10.1 Query 1

Determinare i nomi dei palazzetti in cui non gioca nessuna nazionale asiatica

```
SELECT P.Nome
FROM PALAZZETTO AS P
WHERE P.Nome NOT IN (
    SELECT I.NomePalazzetto
    FROM INCONTRO AS I
    JOIN NAZIONALE AS N1 ON I.Squadra1 = N1.Nazione
    JOIN NAZIONALE AS N2 ON I.Squadra2 = N2.Nazione
    WHERE N1.Continente = 'Asia' OR N2.Continente = 'Asia'
)
```

14.10.2 Query 2

Determinare la capienza complessiva dei palazzetti in cui si giocano partite di nazionali africane (ai fini della valutazione della capienza complessiva, si sommano le capienze associate a ciascuna gara, anche se più gare si svolgono nello stesso palazzetto)

```
-- Subquery = Selezionare i palazzetti dove giocano le nazionali
-- afrincane
-- OuterQuery = Sommatoria le capienze
SELECT SUM(P.Capienza)
FROM PALAZZETTO AS P
JOIN INCONTRO AS I ON P.Nome = I.NomePalazzetto
JOIN NAZIONALE AS N1 ON I.Squadra1 = N1.Nazione
JOIN NAZIONALE AS N2 ON I.Squadra2 = N2.Nazione
WHERE N1.Continente = 'Africa' OR N2.Continente = 'Africa'
```

14.10.3 Query 3

Individuare la città (le città se più di una soddisfa le condizioni dell'interrogazione) in cui si trova il palazzetto in cui la squadra olandese gioca il maggior numero di partite

```
--SubQuery trova il palazzetto dove l'olanda gioca piu partite
--OuterQuery Trova il nome della città di quel palazzetto
SELECT P.città
FROM PALAZZETTO AS P
WHERE COUNT(P.Nome) >= ALL (
    SELECT P1.Nome
    FROM PALAZZETTO AS P1
    JOIN INCONTRO AS I ON I.NomePalazzetto = P1.Nome
    JOIN NAZIONE AS N1 ON I.Squadra1 = N1.Nazione
)
```

```
JOIN NAZIONE AS N2 ON I.Squadra2 = N2.Nazione
WHERE N1.Nazione = 'Olanda' OR N2.Nazione = 'Olanda'
)
```

14.10.4 Query 4

Determinare le squadre che incontrano solo squadre dello stesso livello

```
SELECT N.Nazione
FROM NAZIONALE AS N
WHERE NOT EXISTS (
    SELECT *
    FROM INCONTRO AS I, NAZIONALE AS N2
    JOIN I ON I.Squadra1 = N.Nazione
    JOIN I ON I.Squadra = N2.Nazione
    WHERE N.Livello <> N2.Livello
)
AND
NOT EXISTS (
    SELECT *
    FROM INCONTRO AS I, NAZIONALE AS N2
    JOIN I ON I.Squadra1 = N2.Nazione
    JOIN I ON I.Squadra = N.Nazione
    WHERE N2.Livello <> N.Livello
)
```

14.11 Esercizio 6

Si consideri la seguente Base di Dati:

- DISCO(Nroserie, TitoloAlbum, Anno, Prezzo)
- CONTIENE(NroSerieDisco, CodiceReg, NroProgr)
- ESECUZIONE(CodiceReg, TitoloCanz, Anno)
- AUTORE(Nome, TitoloCanzone)
- CANTANTE(NomeCantante, CodiceReg)

14.11.1 Query 1

La canzone che è stata eseguita almeno due volte

```
SELECT E.TitoloCanz , COUNT(*) AS NumeroEsecuzioni
FROM ESECUZIONE AS E
GROUP BY E.TitoloCanz
HAVING COUNT (*) >= 2
```

14.11.2 Query 2

Gli autori di canzoni eseguite tra il 2000 e il 2010

```
SELECT A.Nome
FROM AUTORE AS A
JOIN ESECUZIONE AS E ON A.TitoloCanzone = E.TitoloCanz
WHERE E.anno >= 2000 AND E.anno <= 2010
```

14.11.3 Query 3

Gli autori delle canzoni più eseguite

```
SELECT A.Nome , A.TitoloCanzone , COUNT(*)
FROM AUTORE AS A
JOIN ESECUZIONE AS E ON E.TitoloCanz = A.TitoloCanzone
GROUP BY A.Nome , A.TitoloCanzone
HAVING COUNT(*) >= ALL (
    SELECT COUNT(*)
    FROM ESECUZIONE
    GROUP BY TitoloCanz
)
```

14.12 Esercizio 7

Data la seguente base di dati

- Concorso(Codice, NumeroProve, Bando)
- Bando(Codice, DataPubblicazione, DataScadenza)
- Candidato(CF, Cognome, Nome, Indirizzo, Telefono)
- Commissario(CF, Cognome, Nome, NumeroTelefono)
- Prova(Concorso, Data, PunteggioMassimo)
- Svolgimento(DataProva, ConcorsoProva, Candidato, Punteggio)
- Partecipazione(Concorso, Candidato, PunteggioTotale, PosizioneInGraduatoria)
- Risposta(Bando, Candidato)
- Commissione(Concorso, Commissario, Ruolo)

14.12.1 Query 1

Selezionare tutti i concorsi del 2018

```
SELECT C.*
FROM Concorso AS C
JOIN Bando AS B ON C.Bando = B.Codice
WHERE (B.DataPubblicazione = '2018-12-1') OR (B.DataScadenza = '2018-12-31')
```

14.12.2 Query 2

Selezionare tutti i candidati che hanno fatto concorsi con bando scaduto nel 2018

```
SELECT C.*
FROM CANDIDATO AS C
JOIN Partecipazione AS P ON C.CF = P.Candidato
JOIN CONCORSO AS CON ON P.Concorso = CON.Codice
JOIN BANDO AS B ON CON.Bando = B.Codice
WHERE YEAR(B.DataScadenza) = 2018
```

14.12.3 Query 3

Trovare i commissari dei concorsi con numero di prove pari a 2

```
SELECT CO.*
FROM COMMISSARIO AS CO
JOIN Commissione AS COM ON CO.CF = COM.Commissario
JOIN CONCORSO AS CON ON COM.Concorso = CON.Codice
WHERE CON.NumeroProve = 2
```

14.13 Esercizio 8

Data la seguente base di dati

- Lavoratore(CF, Nome, Cognome, Età)
- Città(Nome, NumeroAbitanti)
- HaAbitato(CFPersona, Città, DaAnno, AAnno)
- HaLavorato(CFPersona, PIVADitta, Città, DaAnno, AAnno)
- Ditta(PIVA, Nome, NumeroImpiegati, CapitaleSociale)
- HaAvutoSedeIn(PIVADitta, NomeCittà, DaAnno, AAnno)

14.13.1 Query 1

Trovare i lavoratori delle aziende milanesi

```
SELECT L.*  
FROM LAVORATORE AS L  
JOIN HaLavorato AS HL ON HL.CFPersona = L.CF  
WHERE HL.Città = 'Milano'
```

14.13.2 Query 2

Trovare le partite IVA delle ditte che hanno (o hanno avuto) sedi in tutte le città;

```
-- Inserisci qui la query 2
```

14.13.3 Query 3

Per ogni ditta trovare il numero degli impiegati che vi hanno lavorato, risiedendo nella stessa città dove aveva sede la ditta

14.13.4 Query 4

Trovare le città che hanno o hanno avuto il maggior numero di ditte

```
-- Inserisci qui la query 4
```

14.13.5 Query 5

Trovare le città che hanno avuto il maggior numero di ditte con sede nel 2019

```
-- Inserisci qui la query 5
```

14.14 Esercizio 9

- CINEMA(codice, nome, indirizzo, codice_comune)
- COMUNE(codice, nome, provincia, regione, numero_abitanti)
- FILM(codice, titolo, anno, nazionalita, tecnica_animazione)
- PROIETTATO_IN(codice_film, codice_sala, data_ora, lingua)
- SALA(codice, nome, capienza, abilitata_3d, codice_cinema)

14.14.1 Query 1

Selezionare tutti i film di tecnica animazione «computer graphics» proiettati nel 2016

```
SELECT DISTINCT F.*
FROM FILM AS F
JOIN PROIETTATO_IN AS P ON F.codice = P.codice_film
WHERE F.tecnica_animazione = 'computer_graphics' AND YEAR(P.data_ora) = 2016
```

14.14.2 Query 2

Selezionare la capienza media delle sale dei cinema che hanno più di una sala

```
SELECT S.codice_cinema ,AVG(S.capienza)
FROM SALA AS S
GROUP BY S.codice_cinema
HAVING COUNT(*) >= 2
```

14.14.3 Query 3

Selezionare i cinema e nome della città che hanno il maggior numero di posti a vedere.

```
SELECT C.codice, CO.nome, SUM(S.capienza)
FROM CINEMA AS C
JOIN C ON C.codice_comune = CO.codice
JOIN S ON S.codice_cinema = C.codice
GROUP BY C.codice, CO.nome
HAVING SUM(S.capienza) >= ALL (
    SELECT SUM(S2.capienza)
    FROM SALA AS S2
    GROUP BY S2.codice_cinema
)
```